



MIPI CSI-2 to USB UVC

Reference Design

FPGA-RD-02306-1.2

March 2026

Disclaimers

Lattice makes no warranty, representation, or guarantee regarding the accuracy of information contained in this document or the suitability of its products for any particular purpose. All information herein is provided AS IS, with all faults, and all associated risk is the responsibility entirely of the Buyer. The information provided herein is for informational purposes only and may contain technical inaccuracies or omissions, and may be otherwise rendered inaccurate for many reasons, and Lattice assumes no obligation to update or otherwise correct or revise this information. Products sold by Lattice have been subject to limited testing and it is the Buyer's responsibility to independently determine the suitability of any products and to test and verify the same. LATTICE PRODUCTS AND SERVICES ARE NOT DESIGNED, MANUFACTURED, OR TESTED FOR USE IN LIFE OR SAFETY CRITICAL SYSTEMS, HAZARDOUS ENVIRONMENTS, OR ANY OTHER ENVIRONMENTS REQUIRING FAIL-SAFE PERFORMANCE, INCLUDING ANY APPLICATION IN WHICH THE FAILURE OF THE PRODUCT OR SERVICE COULD LEAD TO DEATH, PERSONAL INJURY, SEVERE PROPERTY DAMAGE OR ENVIRONMENTAL HARM (COLLECTIVELY, "HIGH-RISK USES"). FURTHER, BUYER MUST TAKE PRUDENT STEPS TO PROTECT AGAINST PRODUCT AND SERVICE FAILURES, INCLUDING PROVIDING APPROPRIATE REDUNDANCIES, FAIL-SAFE FEATURES, AND/OR SHUT-DOWN MECHANISMS. LATTICE EXPRESSLY DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY OF FITNESS OF THE PRODUCTS OR SERVICES FOR HIGH-RISK USES. The information provided in this document is proprietary to Lattice Semiconductor, and Lattice reserves the right to make any changes to the information in this document or to any products at any time without notice.

Inclusive Language

This document was created consistent with Lattice Semiconductor's inclusive language policy. In some cases, the language in underlying tools and other items may not yet have been updated. Please refer to Lattice's inclusive language [FAQ 6878](#) for a cross reference of terms. Note in some cases such as register names and state names it has been necessary to continue to utilize older terminology for compatibility.

Contents

Contents	3
Figures	5
Abbreviations in This Document.....	7
1. Introduction	8
1.1. Quick Facts	8
1.2. Features	8
1.3. Naming Conventions	9
1.3.1. Nomenclature.....	9
1.3.2. Signal Names	9
2. Directory Structure and Files	10
3. Functional Description.....	11
3.1. Design Components	11
3.1.1. USB23 Controller	11
3.1.2. AHB-Lite to LMMI Converter	11
3.1.3. RISC-V Microcontroller	11
3.1.4. System Memory	12
3.1.5. UART.....	12
3.1.6. GPIO	12
3.1.7. Lattice I2C Controller.....	12
3.1.8. AHB-Lite Interconnect	12
3.1.9. APB Interconnect.....	12
3.1.10. AHB-Lite to APB Converter	12
3.1.11. MIPI CSI/DSI RX IP Core	12
3.1.12. Debayer IP Core.....	13
3.1.13. Color Correction Matrix IP Core	13
3.1.14. Color Space Converter IP Core	13
3.1.15. Chroma Resampler	13
3.1.16. Video to USB Bridge	13
3.1.17. AHB-Lite to Native Memory Bridge	15
3.2. Clocking Scheme	16
3.2.1. Clocking Overview	16
3.3. Reset Scheme	16
3.3.1. Reset Overview	16
4. Signal Description	17
5. Running the Reference Design	18
5.1. Running Propel SDK Project	18
5.2. Running Propel Builder and Radiant Project.....	22
6. Implementing the Reference Design on Board	30
6.1. Programming the FPGA.....	30
6.2. Hardware Setup	33
6.3. Demonstration	34
7. Customizing the Reference Design	38
7.1. Change the Camera Resolution.....	38
7.1.1. Selecting the 1080p Output Resolution	38
7.1.2. Selecting the 720p Output Resolution	40
7.2. Changing IP Configuration to 1 PPC	41
7.2.1. Configuring the MIPI RX IP	41
7.2.2. Configuring the Debayer IP	43
7.2.3. Configuring the Color Correction Matrix IP	45
7.2.4. Configuring the Color Space Converter IP	47
7.2.5. Configuring the Chroma Resampler IP	49
7.2.6. Configuring the Video to USB Bridge.....	51

7.3. Steps to Enable YUV422 Test Pattern	53
8. Resource Utilization	54
9. Debugging	55
9.1. Errors when Updating the SPI Flash	55
9.2. Timing Violations or Place and Route Failed	55
References	57
Technical Support Assistance	58
Revision History	59

Figures

Figure 2.1. Directory Structure	10
Figure 3.1. Reference Design Block Diagram	11
Figure 3.2. Clocking Scheme Overview	16
Figure 3.3. Reset Scheme Overview	16
Figure 5.1. Launch Lattice Propel SDK	18
Figure 5.2. Import an Project in Lattice Propel SDK	19
Figure 5.3. Browse Projects Root Directory	19
Figure 5.4. Propel SDK Project Opened in Lattice Propel SDK	20
Figure 5.5. Navigate the Propel SDK Project	20
Figure 5.6. Set Propel Build Configuration Mode	21
Figure 5.7. Launch the Project Building Process	21
Figure 5.8. Lattice Propel Builder Window	22
Figure 5.9. IP on Local Tab in IP Catalog	23
Figure 5.10. Install IP from File	24
Figure 5.11. IP License Agreement	24
Figure 5.12. IPs Installed from Files	25
Figure 5.13. Mapping the System Memory Initialization Files	26
Figure 5.14. Base Address Assignment	26
Figure 5.15. Validate the Design in Lattice Propel Builder	27
Figure 5.16. Generate in the Lattice Propel Design	27
Figure 5.17. TCL Console Output after Generating the Design	27
Figure 5.18. Lattice Radiant Icon in Lattice Propel Builder	27
Figure 5.19. Lattice Radiant Software Window	28
Figure 5.20. Synthesis Strategy Settings	28
Figure 5.21. Place and Route Design Strategy Settings	29
Figure 5.22. Generating Bitstream	29
Figure 5.23. Compilation Completed	29
Figure 6.1. Creating a New Project from a Scan	30
Figure 6.2. Lattice Radiant Programmer Window	30
Figure 6.3. CrossLinkU-NX Device Settings	31
Figure 6.4. Device Properties Window for the CrossLinkU-NX SPI Flash Programming	31
Figure 6.5. Programmer Menu Bar	32
Figure 6.6. Programming Output Window	32
Figure 6.7. CrossLinkU-NX Evaluation Board Setup for Demonstration	33
Figure 6.8. USB Enumeration	33
Figure 6.9. Open Webcam or Other Devices in PotPlayer	34
Figure 6.10. Video Output on PotPlayer	35
Figure 6.11. Video Renderer Option	35
Figure 6.12. Playback/System Information	36
Figure 6.13. Example of Video Information	37
Figure 7.1. Device Settings in PotPlayer	39
Figure 7.2. 1080p Selection in PotPlayer	39
Figure 7.3. Device Settings in PotPlayer	40
Figure 7.4. 720p Selection in PotPlayer	41
Figure 7.5. MIPI RX IP Configuration	42
Figure 7.6. MIPI RX IP Check Generated Result Window	43
Figure 7.7. Debayer Configuration	44
Figure 7.8. Debayer IP Check Generated Result Window	45
Figure 7.9. Color Correction Matrix Configuration	46
Figure 7.10. Color Correction Matrix IP Check Generated Result Window	47
Figure 7.11. Color Space Converter Configuration	48
Figure 7.12. Color Space Converter IP Check Generated Result Window	49

Figure 7.13. Chroma Resampler Configuration	50
Figure 7.14. Chroma Resampler IP Check Generated Result Window	51
Figure 7.15. Video-to-USB Configuration	52
Figure 7.16. Video-to-USB Check Generated Result Window	53
Figure 7.17. Firmware Code to Enable the YUV422 Test Pattern	53
Figure 9.1. TCK Frequency Settings	55
Figure 9.2. Port Selection.....	55
Figure 9.3. Place and Route Failed Message	55
Figure 9.4. Place and Route Design Strategy Settings	56

Tables

Table 1.1. Summary of the Reference Design	8
Table 2.1. File List	10
Table 3.1. Video to USB Bridge Parameters	13
Table 3.2. Video to USB Bridge Signals	13
Table 4.1. Primary I/O.....	17
Table 7.1. IP Configuration for Supported Resolutions	38
Table 8.1. Resource Utilization for CrosslinkU-NX MIPI CSI-2 to USB UVC Reference Design.....	54

Abbreviations in This Document

A list of abbreviations used in this document.

Abbreviations	Definition
APB	Advanced Peripheral Bus
AXI4	Advanced eXtensible Interface 4
BPC	Bits per component
CCM	Color correction matrix
CSI-2	Camera Serial Interface-2
DMA	Direct Memory Access
DSI	Display Serial Interface
FIFO	First In First Out
FPS	Frame per second
HARM	High-risk uses
IEBM	In Endpoint Buffer Manager
IP	Intellectual Property (IP Core)
ISP	Image signal processing
J2	Micro USB port
LMMI	Lattice Memory Mapped Interface
PC	Personal Computer
PPC	Pixels per Clock
RAM	Random Access Memory
RISC-V	Reduced Instruction Set Computer – Five
RGB	Red, Green, Blue
SCL	I2C controller serial clock line
SDA	I2C controller serial data line
TDP	True Dual-Port
USB	Universal Serial Bus
UVC	USB Video Class
UVSI	Unified Video Streaming Interface

1. Introduction

The Lattice Semiconductor CrosslinkU™-NX Mobile Industry Processor Interface (MIPI®) Camera Serial Interface-2 (CSI-2) to Universal Serial Bus (USB) Video Class (UVC) reference design provides a template for unified video streaming from a camera sensor, utilizing the USB hard IP in a CrosslinkU-NX device.

1.1. Quick Facts

Download the MIPI CSI-2 to USB UVC Reference Design's design files from the corresponding [GitHub page](#).

Table 1.1. Summary of the Reference Design

General	Target Devices	LIFCL-33U
	Source code format	C Code, Verilog and SystemVerilog RTL
Simulation	Functional simulation	Not performed
	Timing simulation	Not performed
	Test bench	Not available
	Test bench format	Not applicable
Software Requirements	Software tool and version	Lattice Propel™ SDK 2025.2 Lattice Propel Builder 2025.2 Lattice Radiant™ Software 2025.2 Lattice Radiant Programmer 2025.2 PotPlayer Software (3 rd Party) (Download from Microsoft Store or Potplayer Official Website, version 1.7)
	IP version (if applicable)	MIPI CSI/DSI RX IP Core version 1.0.0.02 (Custom IP) Color Space Converter IP Core version 2.4.1.01 (Custom IP) Color Correction Matrix IP Core version 1.3.1.01 (Custom IP) Debayer IP Core version 1.4.0.01 (Custom IP) FIFO_DC IP version 2.3.0 USB23 IP Core version 3.1.1
Hardware Requirements	Board	LIFCL-33U-EVN Evaluation Board REV-B
	Camera	Raspberry Pi Camera Module V2
	Cable	USB C to USB C cable, or USB C to USB A 9-pin cable USB A to Micro USB cable

1.2. Features

Key features of the MIPI CSI-2 to USB UVC reference design include:

- The Lattice MIPI CSI/DSI RX IP Core in this reference design is configured to support two lanes (capped by IMX219 capability) with lane rate of 592 Mbps (capped by driver configuration). The IP receives, processes, and converts incoming MIPI video payload packets into Unified Video Streaming Interface (UVSI) packets. For details, refer to <Project Directory>/latticesemi_custom_ip_mipi_csi_dsi_1.0.0.02/doc/IP_Design_Notes.pdf.
- The Lattice Debayer IP Core extracts the R, G, and B components from the pixel data output by the image sensor, converting the RAW10 data format into full RGB components. For details, refer to <Project Directory>/latticesemi_custom_ip_debayer_1.4.0.01/doc/IP_Design_Notes.pdf
- The Lattice Color Correction Matrix IP Core performs pixel data correction by adjusting R, G, and B components gain and compensates for color channel crosstalk. For details, refer to <Project Directory>/latticesemi_custom_ip_ccm_1.3.1.01/doc/IP_Design_Notes.pdf.
- The Color Space Converter IP Core converts RGB data into the YUV format. For details, refer to <Project Directory>/latticesemi_custom_ip_color_space_converter_2.4.1.01/doc/IP_Design_Notes.pdf
- The Chroma Resampler converts YUV444 data into the YUV422 format by reducing the chroma component.
- The Video to USB bridge takes in UVSI packets and form data stream in USB Video Class (UVC) format for USB transfer.

- The hardware USB IP Core streams the unified video data from the image sensor to a PC through a USB Type-C connector.

1.3. Naming Conventions

1.3.1. Nomenclature

The nomenclature used in this document is based on Verilog HDL and SystemVerilog HDL.

1.3.2. Signal Names

- `_n` are active low, asserted when value is logic 0.
- `_i` are input signals.
- `_o` are output signals.
- `_io` are bi-directional input and output signals.

2. Directory Structure and Files

Figure 2.1 shows the directory structure.

```
+---docs //Contains reference design user guide
+---fpga_lifcl //FPGA_DEVICE_FAMILY
| +---precompiled_file //Contains a precompiled bitstream (IP Evaluation) for the default configuration
+---propelsdk
|   +---riscv_mc
\---radiant //Contains Lattice Propel Builder files, Radiant files and RTL source code
    +---propelbld_mipi_csi_to_usb_uvc
    |   \---lib
    |       +---gluelogics
    |       +---IPs
    |       \---latticesemi.com
    |           +---ip
    |               +---ahbl_to_axilite_bridge
    |               +---ahbl_to_lmml_inst
    |               +---ahbl_to_mem_inst
    |               +---cpu0
    |               +---HW_ver_gpio
    |               +---lsc_i2cc
    |               +---system0
    |               +---tp_en_gpio
    |               +---uart0
    |           \---module
    |               +---ahbl0
    |               +---ahbl2apb0
    |               +---apb0
    |               +---pll_multiclk
    |           \---USB23
    |       \---latticesemi_custom
    |           +---ip
    |               +---isp_ccm
    |               +---isp_chroma_resampler
    |               +---isp_csc
    |               +---isp_debayer
    |               +---mipi_csi_rx
    |               +---unified_video_to_usb
    |               +---usb_bus_interface_bridge
    |       +---sge //System Generation Environment (Generated by Propel Builder)
    |       +---src
    |           \---constraints //Contains the constraints file (.pdc) used in the design
    |       \---verification //Generated by Propel Builder
+---misc //Contains non-FPGA files
|   +---custom_ip //Contains Lattice IP Packagers for custom IPs (eg: ISP IP, AHBL to AXI4-Lite, and etc)
```

Figure 2.1. Directory Structure

Table 2.1 shows the list of files included in the reference design package.

Table 2.1. File List

Attribute	Description
<Component name>.ipx	This file contains the information on the files associated with the generated IP.
<Component name>.cfg	This file contains the parameter values used in IP configuration.
component.xml	Contains the ipxact:component information of the IP.
design.xml	Documents the configuration parameters of the IP in IP-XACT 2014 format.
rtl/<Component name>.v	This file provides an example RTL top file that instantiates the module.
rtl/<Component name>_bb.v	This file provides the synthesis closed box.
misc/<Component name>_tmpl.v misc /<Component name>_tmpl.vhd	These files provide instance templates for the module.

The top-level block diagram of the CrosslinkU-NX MIPI CSI-2 to USB UVC reference design is shown in [Figure 3.1](#) below.



The CrosslinkU-NX MIPI CSI-2 to USB UVC reference design includes blocks described in the following sections. In this reference design, video data is captured through the CSI-2 interface using the RAW10 color format. The incoming RAW10 data stream is converted to RGB888, subsequently YUV444, and finally YUV422 through the image signal processing (ISP) pipeline. The ISPs operate with a configuration of 8 bits per component (BPC) and 2 pixels per clock (PPC), with a maximum supported video resolution of 1080p30.

The AXI4 Direct Memory Access (DMA) Controller of the USB23 Controller is connected to two different memories through the AXI4 Interconnect Bridge.

- For Bulk transfer, the USB controller fetches data from the In Endpoint Buffer Manager (IEBM) module's data Random Access Memory (RAM). This RAM is not shared with any other controller and/or RISC-V processor.
- For all other operations, the USB controller talks to the system memory, which is shared with the RISC-V processor. This kind of architecture helps to achieve higher Bulk throughput.

The USB23 controller register interface can be configured through its Lattice Memory Mapped Interface (LMMI). Since RISC-V MC supports only the AHB-Lite interface, we need an AHB-Lite to LMMI converter.

All the serial peripheral blocks are connected to the RISC-V MC soft processor through the AHB-Lite interface. The RISC-V MC CPU IP processes data and instructions while monitoring external interrupts coming from the USB23 controller and JEBM core.

3.1.4. System Memory

This IP is a soft IP for memory which is used by the RISC-V MC processor. The processor reads the instructions stored in this memory through the AHB-Lite interface and takes actions accordingly. This IP has two interfaces, AHB-Lite and AXI4, through which the manager can communicate with it. The AHB-Lite interface is used to communicate with the RISC-V processor.

3.1.5. UART

The RISC-V MC processor can communicate with the UART and GPIO cores through its Advanced Peripheral Bus (APB) interface. Hence, the UART and GPIO cores are connected to the soft processor through the AHB-Lite to APB Bridge. The UART IP performs serial-to-parallel conversion on data characters received from a peripheral UART device. The IP performs parallel-to-serial conversion on data characters received from the host through an APB interface.

3.1.6. GPIO

There is a 32-bit input GPIO, which is the hardware version GPIO, to monitor the FPGA design version for a particular release. Also, there is a 2-bit output GPIO, known as the test pattern enable GPIO. This GPIO is used to select the data either from the camera sensor RX DPHY or MIPI RAW10 Test Pattern Generator, and Image Signal Processing stream or YUV422 Test Pattern Generator.

3.1.7. Lattice I2C Controller

This Lattice I2C Controller IP is designed for communicating with any I2C target device. It supports standard operations such as I2C write, I2C read, and combined write-then-read transactions with the connected device. In this design, the RISC-V MC processor uses the I2C Controller IP to configure the registers of the MIPI camera sensor, enabling proper initialization and operation of the camera.

3.1.8. AHB-Lite Interconnect

This IP provides the communicating interface between RISC-V soft processor and peripherals that support AHB-Lite interface, such as System Memory and AHB-Lite to LMMI Bridge.

3.1.9. APB Interconnect

This IP provides the communicating interface between the RISC-V MC soft processor and other slow peripherals that support the APB interface, such as UART and GPIO.

3.1.10. AHB-Lite to APB Converter

The AHB-Lite to APB Bridge provides an interface between the AHB-Lite manager and APB completer. Read and write transfers on AHB are converted into equivalent transfers on APB.

3.1.11. MIPI CSI/DSI RX IP Core

The Lattice MIPI CSI/DSI D-PHY IP provides a flexible solution for receiving high-speed video data over a MIPI-compliant D-PHY interface in FPGA designs. MIPI enables high-speed, low-power video transmission between components such as cameras and displays. In this reference design, the video packets are received through D-PHY physical layer interface from the Raspberry Pi Camera Module V2. The incoming video payload packets are processed and converted from bytes to pixels before being transmitted through UVSI. For complete specifications, including interface signals and parameter details, refer to the MIPI CSI/DSI RX IP documentation:

< Project Directory>/latticesemi_custom_ip_mipi_csi_dsi_1.0.0.02/doc/IP_Design_Notes.pdf

3.1.12. Debayer IP Core

The image sensor outputs pixel data in Bayer format. To extract the individual R, G, and B color components from this Bayer-patterned data, a Debayer function is required. The Debayer IP converts RAW10 image data into full RGB components. This IP supports UVS1 interfaces for both its input and output video streams. For complete specifications, including interface signals and parameter details, refer to the Debayer IP documentation:

<Project Directory>/latticesemi_custom_ip_debayer_1.4.0.01/doc/IP_Design_Notes.pdf

3.1.13. Color Correction Matrix IP Core

To obtain the correct colors, the IP performs pixel data correction by adjusting R, G, and B components gain and compensating for color channel crosstalk. This linear mapping of the color components is achieved by using a 3x3 matrix, called color correction matrix (CCM). For complete specifications, including interface signals and parameter details, refer to the Color Correction Matrix IP documentation:

<Project Directory>/latticesemi_custom_ip_ccm_1.3.1.01/doc/IP_Design_Notes.pdf

3.1.14. Color Space Converter IP Core

The Color Space Converter IP block converts RGB data into the YUV format. For complete specifications, including interface signals and parameter details, refer to the Color Space Converter IP documentation:

<Project Directory>/latticesemi_custom_ip_color_space_converter_2.4.1.01/doc/IP_Design_Notes.pdf

3.1.15. Chroma Resampler

The Chroma Resampler converts YUV444 data into the YUV422 format by reducing the chroma component.

3.1.16. Video to USB Bridge

The Video to USB Bridge is responsible for receiving video data from UVS1, packing into UVC-compliant format, buffering using IN Endpoint Buffer Manager (IEBM) and True Dual-Port (TDP) Memory, and handling the transfer between data buffer and USB23 controller for host transmission. In addition, the Video to USB Bridge features a YUV422 Test Pattern Generator (TPG) for debugging available for use by enabling it using the control plane.

To support the USB data flow, the bridge uses an IEBM to organize and store video data before it reaches the USB23 controller. The IEBM writes video data sequentially into endpoint buffers through a FIFO interface and then exposes these buffers to the USB23 controller through AXI4 for reading. This ensures continuous and orderly movement of video frames through the system. The TDP Memory offers dedicated storage that supports simultaneous read and write operations, helping maintain stable video throughput even at high data rates.

This module enables seamless data flow from UVS1 to USB controller while being easy-to-use, minimizing design blocks required to be instantiated by the user to achieve the transfer.

Refer to [Table 3.1](#) and [Table 3.2](#) below for parameters and signals description of this module.

Table 3.1. Video to USB Bridge Parameters

Port Name	Selectable Values	Default	Description
COLOR_FORMAT	YUV422	YUV422	Color Format
BPC	8	8	Bits Per Component
PIXEL_PER_CLK	1 or 2 or 4	4	Pixel Per Clock

Table 3.2. Video to USB Bridge Signals

Port Name	I/O	Width	Description
pixel_clk	Input	1	Pixel Clock
clk_pll_72m	Input	1	72 MHz Clock generated by PLL
mipi_byte_clk	Input	1	MIPI Byte Clock
system_reset_n	Input	1	AHB-Lite reset (Active Low)
pixel_clk_reset_n	Input	1	Reset for logic operating in the pixel clock domain (Active Low)

Port Name	I/O	Width	Description
mipi_byte_clk_resetrn	Input	1	Reset for MIPI byte-clock domain logic (Active Low)
yuy2_tp_en	Input	1	Enables the internal YUY2 (YUV422) test-pattern generator through GPIO.
Native Memory Interface for True Dual-Port Memory			
mem_to_ahbl_32_wr_en_b	Input	1	Write-enable signal for writing data
mem_to_ahbl_32_wr_data_b	Input	[13:0]	Write-data bus for buffered memory writes
mem_to_ahbl_32_ben_b	Input	[3:0]	Byte-enable signals for selecting active write bytes
mem_to_ahbl_32_addr_b	Input	[9:0]	Address bus for memory transactions
mem_to_ahbl_32_rd_data_b	Output	[31:0]	Read-data output
AHB-Lite Subordinate Interface for IEBM			
ahbl_haddr_i	Input	[7:0]	AHB-Lite address bus for subordinate access.
ahbl_hburst_i	Input	[2:0]	Burst-type indicator for AHB-Lite transactions.
ahbl_hmastlock_i	Input	1	Manager-lock signal indicating locked AHB sequences
ahbl_hprot_i	Input	[3:0]	Protection control signals for privilege/cache/buffer settings
ahbl_hready_i	Input	1	Indicates that the previous transfer is complete. It allows new transfers.
ahbl_hsel_i	Input	1	AHB-Lite subordinate select signal
ahbl_hsize_i	Input	[2:0]	Specifies the transfer size, byte, half-word, or word.
ahbl_htrans_i	Input	[1:0]	AHB-Lite transfer type
ahbl_hwdata_i	Input	[31:0]	Write data from AHB-Lite manager
ahbl_hwrite_i	Input	1	Write direction indicator. 1=WRITE, 0=READ.
ahbl_hrdata_o	Output	[31:0]	Read data returned to AHB-Lite manager
ahbl_hreadyout_o	Output	1	Indicates the subordinate is ready to complete the current transfer.
ahbl_hresp_o	Output	1	Transfer response
AXI4 Subordinate Interface for USB23 Controller			
u23_axim_XMAWADDR	Input	[31:0]	AXI4 write-address channel: write address
u23_axim_XMAWBURST	Input	[1:0]	Burst type for write transactions
u23_axim_XMAWID	Input	[7:0]	Write-transaction ID
u23_axim_XMAWLEN	Input	[7:0]	Number of beats in a write burst
u23_axim_XMAWPROT	Input	[2:0]	Protection bits for write transactions
u23_axim_XMAWSIZE	Input	1	Write data size per beat (encoded)
u23_axim_XMAWREADY	Output	1	Indicates acceptance of write address.
u23_axim_XMAWVALID	Input	1	Write-address valid signal
u23_axim_XMWDATA	Input	[63:0]	Write-data bus
u23_axim_XMWSTRB	Input	[7:0]	Write strobe bits per byte
u23_axim_XMWLAST	Input	1	Indicates last beat of write burst
u23_axim_XMWREADY	Output	1	Indicates acceptance of write data
u23_axim_XMWVALID	Input	1	Write-data valid signal
u23_axim_XMARADDR	Input	[31:0]	AXI4 read-address channel: read address
u23_axim_XMARBURST	Input	[1:0]	Burst type for read transactions
u23_axim_XMARID	Input	[7:0]	Read-transaction ID
u23_axim_XMARLEN	Input	[7:0]	Number of beats in a read burst
u23_axim_XMARPROT	Input	[2:0]	Protection bits for read transactions
u23_axim_XMARSIZE	Input	[2:0]	Read data size per beat
u23_axim_XMARREADY	Output	1	Indicates acceptance of read address.
u23_axim_XMARVALID	Input	1	Read-address valid signal
u23_axim_XMRDATA	Output	[63:0]	Read-data bus

Port Name	I/O	Width	Description
u23_axim_XMRID	Output	[7:0]	ID of the read-data response
u23_axim_XMRRESP	Output	[1:0]	Read response type
u23_axim_XMRLAST	Output	1	Indicates the last beat of a read burst.
u23_axim_XMRREADY	Input	1	Indicates the Manager is ready to accept read data.
u23_axim_XMRVALID	Output	1	Read-data valid signal
u23_axim_XMBID	Output	[7:0]	ID for write-response channel
u23_axim_XMBRESP	Output	[1:0]	Write response
u23_axim_XMBREADY	Input	1	Indicates the Manager is ready for write response.
u23_axim_XMBVALID	Output	1	Write-response valid
Unified Video Streaming Interface			
tdata	Input	[PIXEL_PER_CLK*16-1:0]	TDATA is the primary payload that is used to provide the data that is passing across the interface.
tuser	Input	[1:0]	Sideband information is transmitted alongside the data stream. tuser[0] – start of frame tuser[1] – interlaced video
tlast	Input	1	TLAST indicates the end of the line for each horizontal line. It always coincides with the last pixel of the line.
tready	Output	1	TREADY indicates that RX can accept a transfer in the current cycle.
tvalid	Input	1	TVALID indicates availability of valid pixel data on tx_tdata_o when TREADY is asserted.
Interrupt			
iebm_ri_irq_o	Output	1	Interrupt request generated by IEBM to RISC-V indicating buffer ready to read.

3.1.17. AHB-Lite to Native Memory Bridge

This block helps RISC-V data manager to access the True Dual-Port (TDP) memory, which is dedicated for USB operations. This block converts the AHB-Lite subordinate signal into the native memory controller interface signals.

3.2. Clocking Scheme

The CrosslinkU-NX MIPI CSI-2 to USB UVC Reference Design uses a PLL to generate the 72 MHz system clock, 120 MHz pixel clock for image signal processing, and 60 MHz USBPHY clock. All these clocks are generated from the 60 MHz crystal oscillator on LIFCL-33U Evaluation Board.

3.2.1. Clocking Overview

An overview of the CrosslinkU-NX MIPI CSI-2 to USB UVC Reference Design clocking scheme is shown in [Figure 3.2](#).

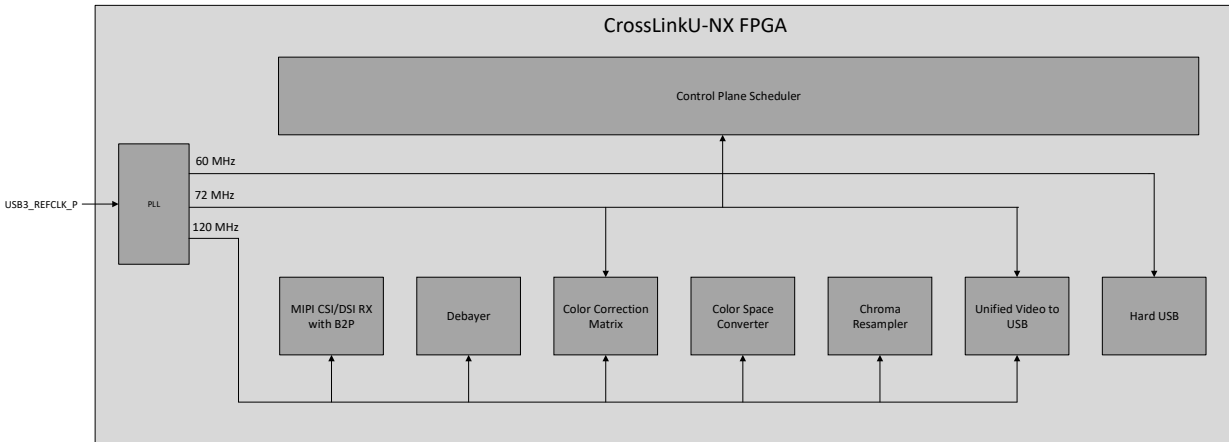


Figure 3.2. Clocking Scheme Overview

3.3. Reset Scheme

Synchronize resets are generated for each clock domain from the PLL lock signal and corresponding PLL clock outputs.

3.3.1. Reset Overview

An overview of the CrosslinkU-NX MIPI CSI-2 to USB UVC Reference Design reset scheme is shown in [Figure 3.3](#).

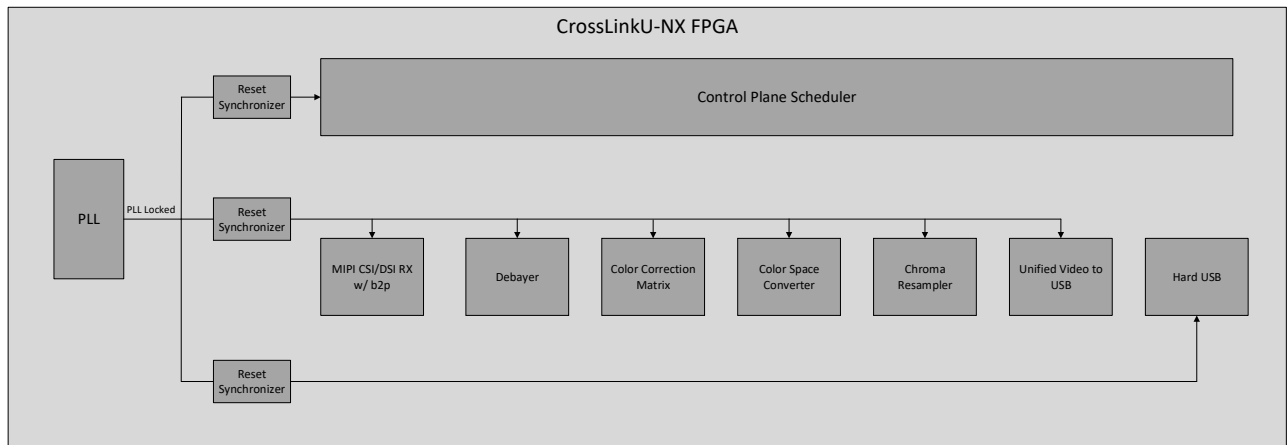


Figure 3.3. Reset Scheme Overview

4. Signal Description

Table 4.1 shows the input and output interface signals for this reference design.

Table 4.1. Primary I/O

Port Name	I/O	Width	Description
clk_60m_i	Input	1	60 MHz input clock from the on-board crystal oscillator.
dp_z	Inout	1	USB 2.0 positive differential data line
dm_z	Inout	1	USB 2.0 negative differential data line
u3_rxm_i	Input	1	USB 3.0 positive differential data line for the receiver
u3_rxp_i	Input	1	USB 3.0 negative differential data line for the receiver
vbus_z	Inout	1	Power signal
u2_reset_ext_z	Inout	1	Synchronous reset for USB 2.0
REFINCLKEXTTP_i	Input	1	USB 3.0 PHY external positive differential clock
REFINCLKEXTM_i	Input	1	USB 3.0 PHY external negative differential clock
u3_txm_o	Output	1	USB 3.0 positive differential data line for the transmitter
u3_txp_o	Output	1	USB 3.0 negative differential data line for the transmitter
cam_scl_z	Inout	1	I2C controller serial clock line (SCL) for the camera sensor
cam_sda_z	Inout	1	I2C controller serial data line (SDA) for the camera sensor
cam_en_o	Output	1	Camera sensor enable
rx_clk_p_i	Inout	1	Camera positive differential clock line
rx_clk_n_i	Inout	1	Camera negative differential clock line
rx_d0_p_i	Inout	1	Camera positive differential data0 line
rx_d0_n_i	Inout	1	Camera negative differential data0 line
rx_d1_p_i	Inout	1	Camera positive differential data1 line
rx_d1_n_i	Inout	1	Camera negative differential data1 line
uart_rxd_i	Input	1	UART input for RISC-V microcontroller debugging
uart_txd_o	Output	1	UART output for RISC-V microcontroller debugging

5. Running the Reference Design

This section describes the steps to run the reference design in the Lattice Propel SDK, Lattice Propel Builder and Lattice Radiant software.

5.1. Running Propel SDK Project

This reference design utilizes the hardened USB block inside the LIFCL-33U device, which needs to be configured and performs USB enumeration through the RISC-V firmware. This section describes how to build and run a C project in the Lattice Propel SDK software.

To run the reference design in the Lattice Propel SDK software, follow these steps:

1. Launch the Lattice Propel SDK software.
2. Click the **Browse** button to set the workspace folder to <Project Directory>\RD02306_mipi_csi_to_usb_uvc\propelsdk.
3. Then, click the **Launch** button, as shown in [Figure 5.1](#).

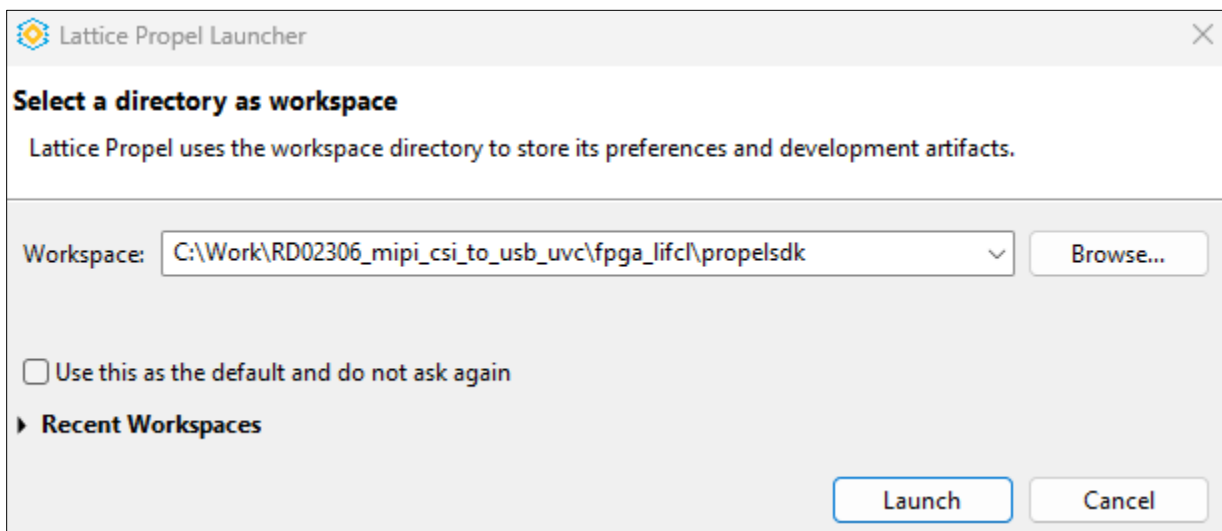


Figure 5.1. Launch Lattice Propel SDK

4. In the GUI, click **Import projects...**, and then select **Existing Projects into Workspace** under **General**.

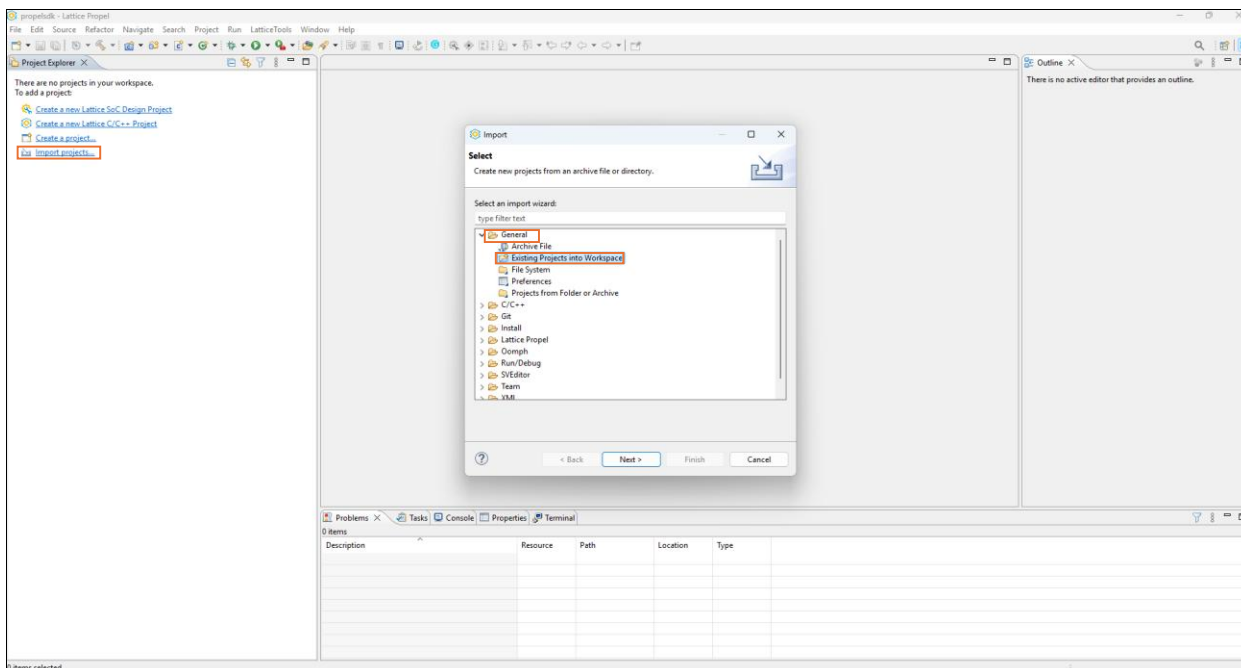


Figure 5.2. Import an Project in Lattice Propel SDK

5. Click the **Browse** button to set the root directory:
`<Project Directory>\RD02306_mipi_csi_to_usb_uvc\propelsdk\riscv_mc`.
 Then, click Finish.

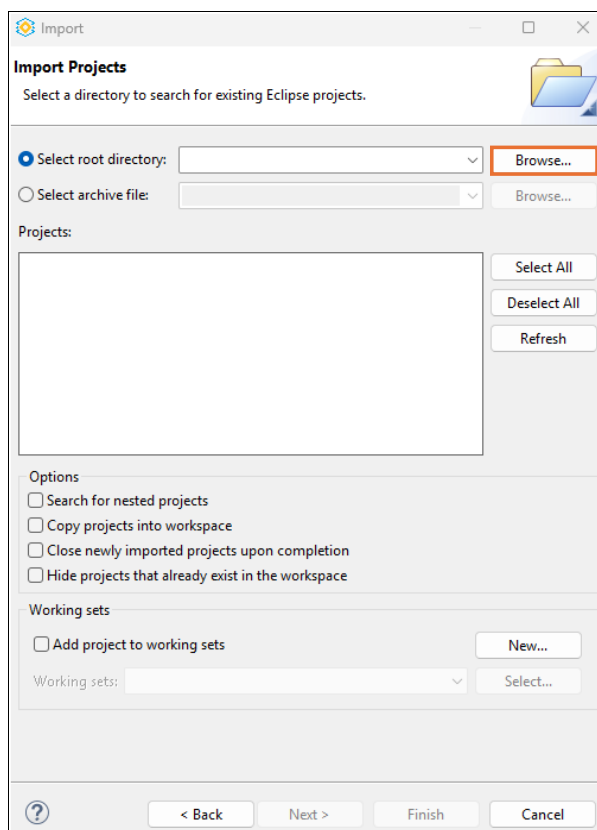


Figure 5.3. Browse Projects Root Directory

6. The Propel SDK project is opened and you can see the riscv_mc project (Figure 5.4).

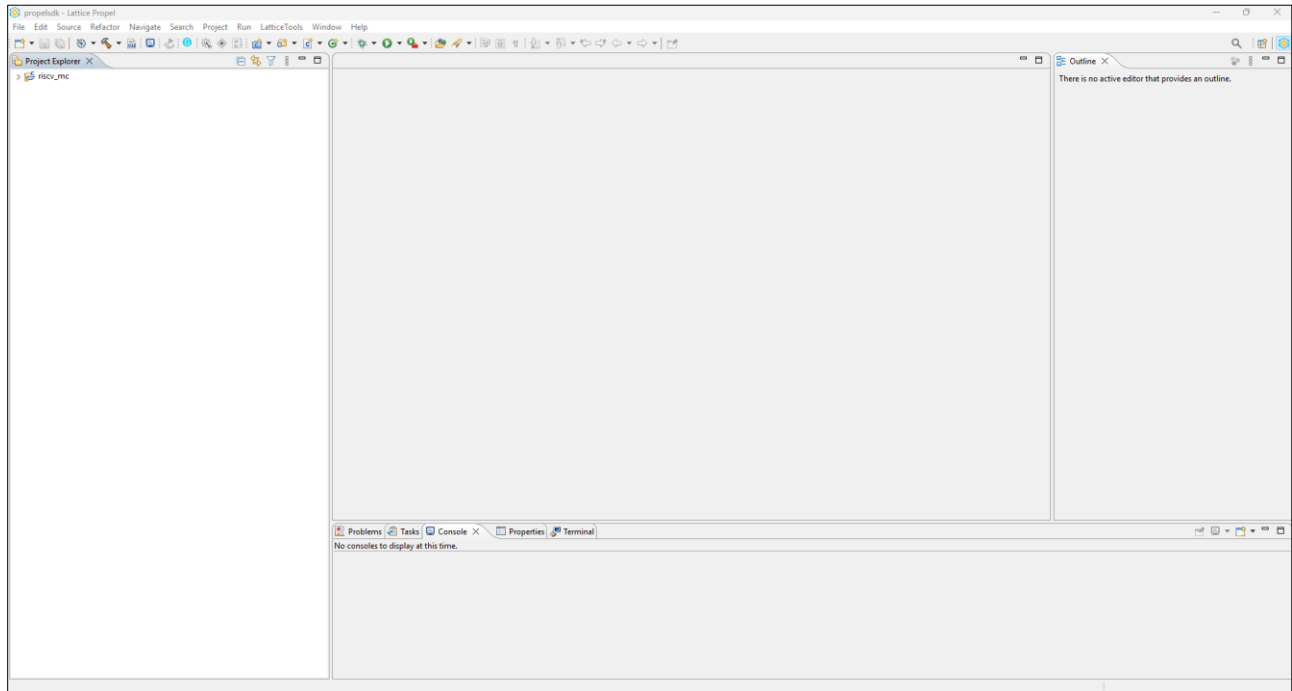


Figure 5.4. Propel SDK Project Opened in Lattice Propel SDK

7. Navigate to **riscv_mc > src** to expand the project file list, as shown in Figure 5.5.

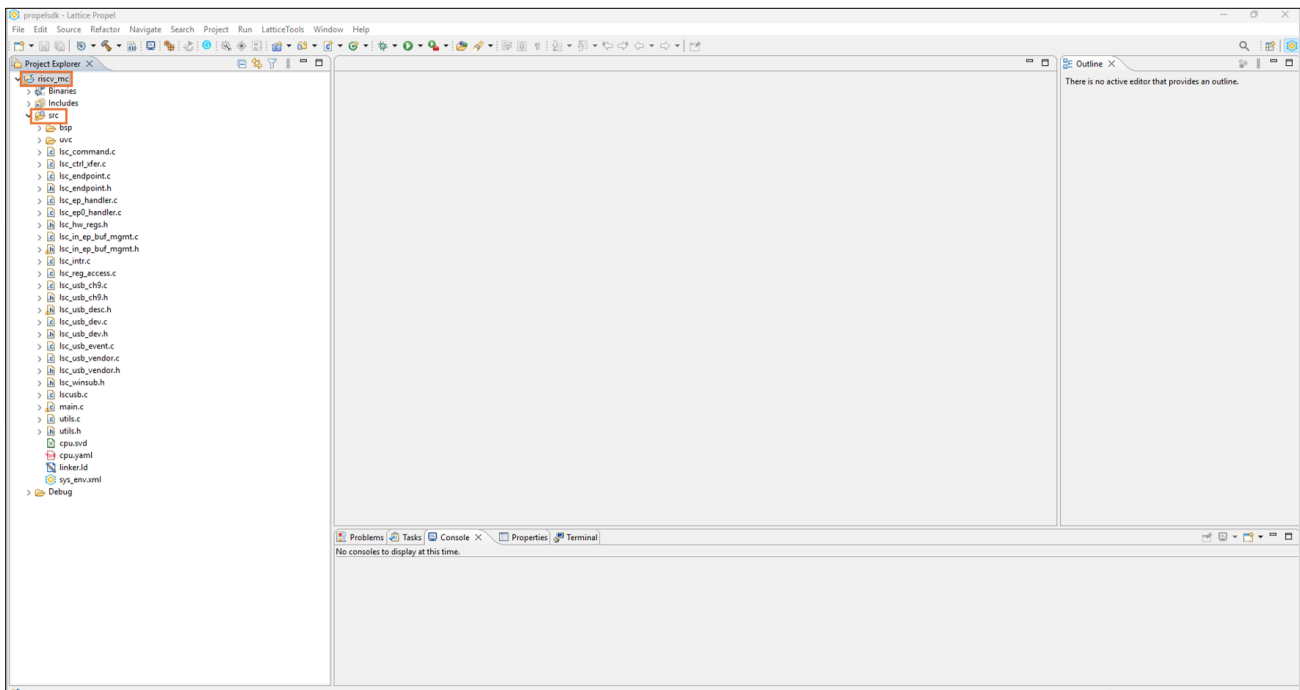


Figure 5.5. Navigate the Propel SDK Project

8. Select **Project > Build Configurations > Set Active > 1 Debug** (Figure 5.6) to compile the Propel SDK project to generate the memory image for the RISC-V firmware.

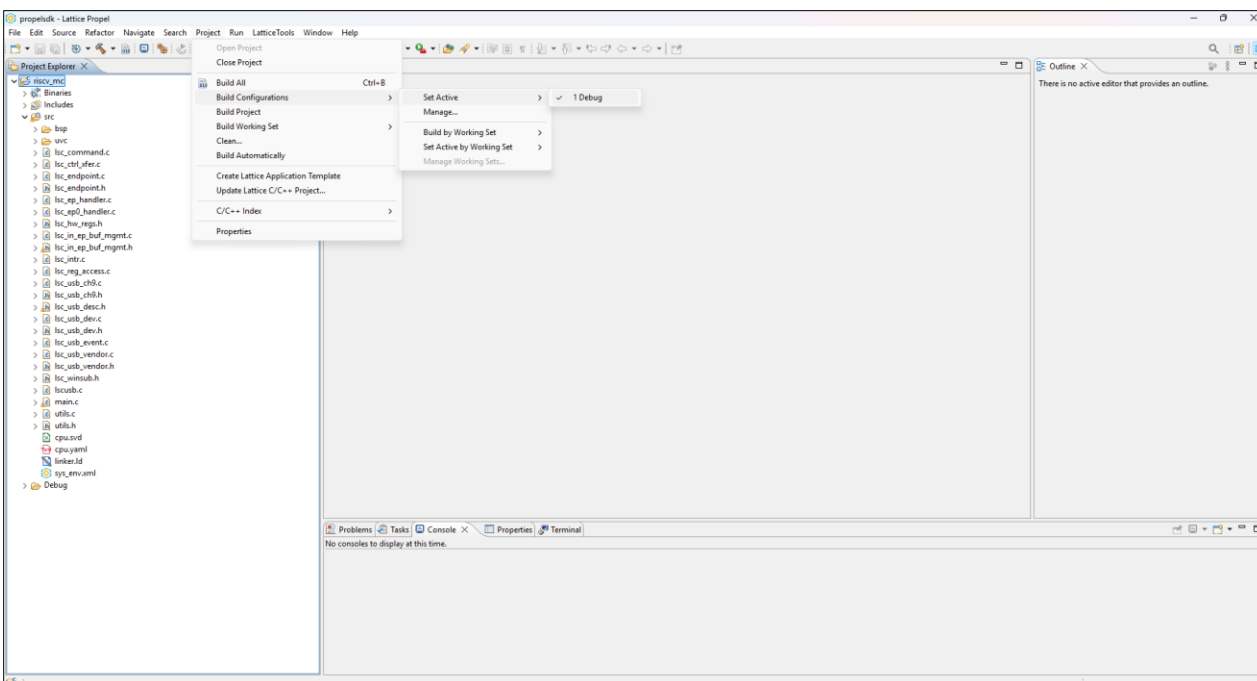


Figure 5.6. Set Propel Build Configuration Mode

- To launch the project building process, right-click on the process name **riscv_mc**, select **Clean Project** from the menu to clean up the output folders. Then, select **Build Project** from the menu, as shown in Figure 5.7. The output **riscv_mc.mem** file is located at:
`<Project Directory>\RD02306_mipi_csi_to_usb_uvc\propelsdk\riscv_mc\Debug\riscv_mc.mem.`

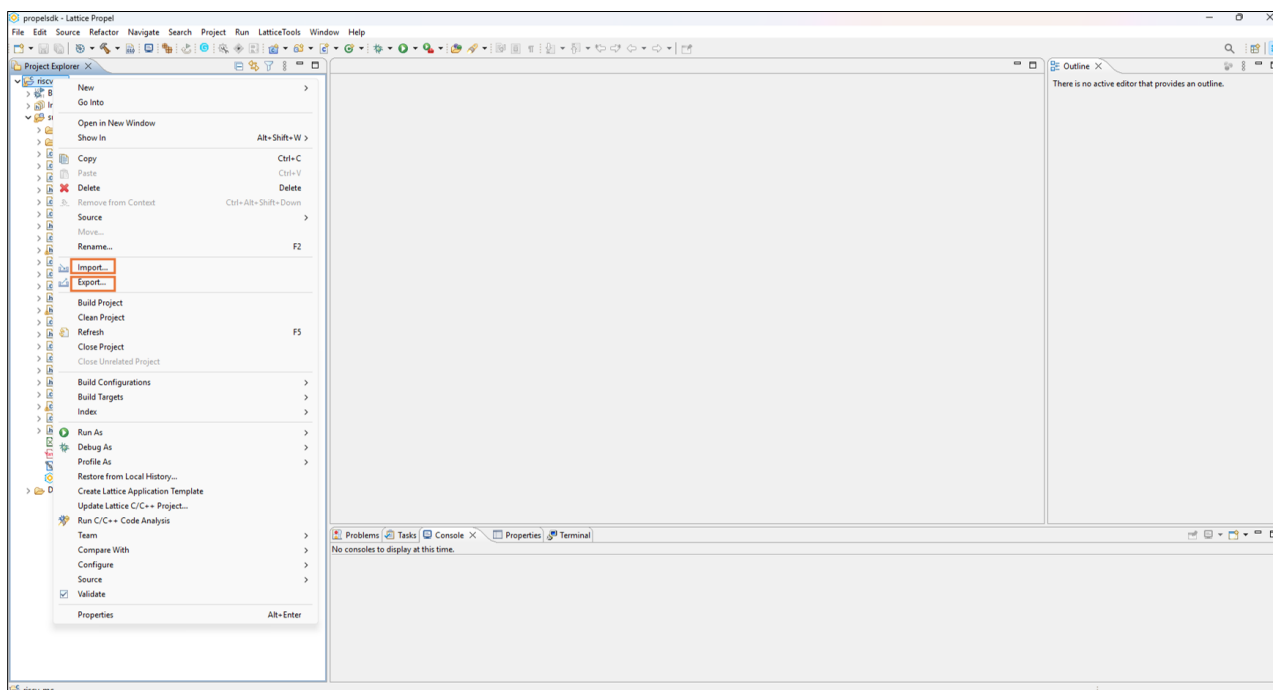


Figure 5.7. Launch the Project Building Process

5.2. Running Propel Builder and Radiant Project

This section describes the steps to run the reference design in the Lattice Propel Builder and Lattice Radiant software.

To run the reference design in the Lattice Propel Builder and Lattice Radiant software, follow these steps:

1. Launch the Lattice Propel Builder software.
2. Go to **File > Open Design**, browse and select the existing project file: *mipi_csi_to_usb_uvc.sbx*. The design appears, as shown in Figure 5.8.

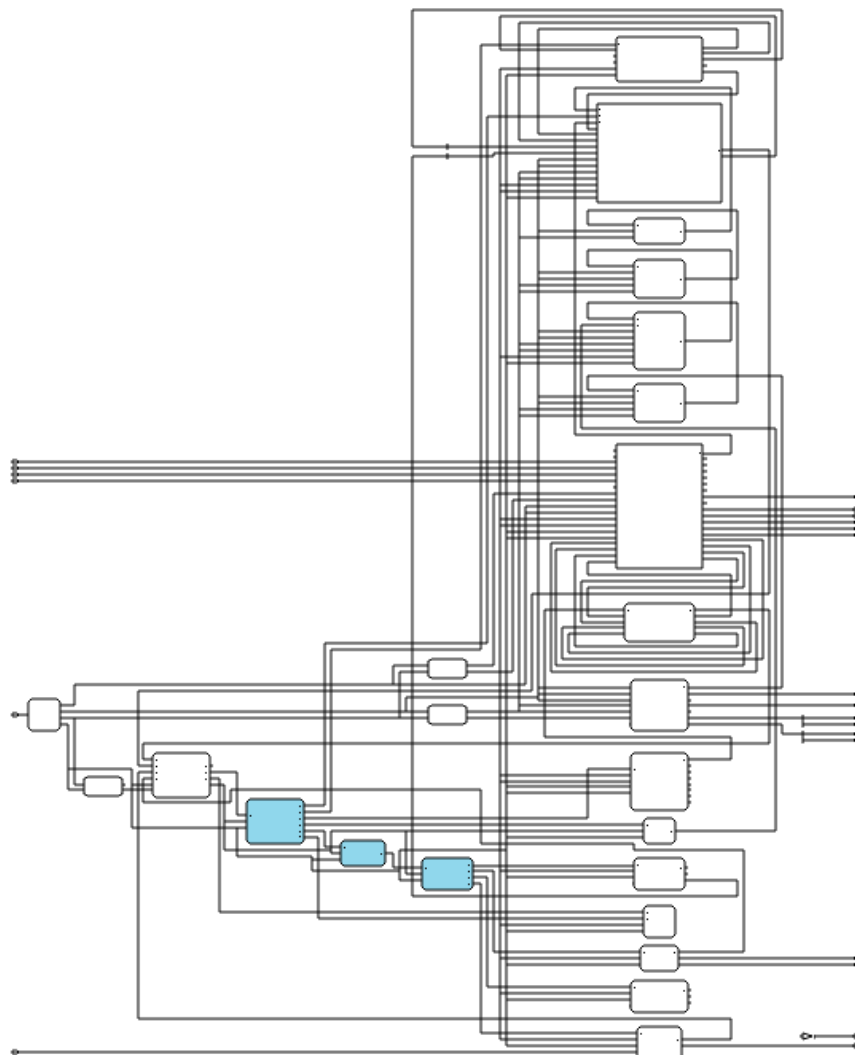


Figure 5.8. Lattice Propel Builder Window

3. Before performing any IP reconfiguration, ensure that the IP has been downloaded and installed under IP on Local, as shown in Figure 5.9. In this reference design, there are few IPKs that need to be installed into IP on Local. Follow steps below to perform the IPK installation.
 - a. Right-click **IP on Local** and select **Install IP from file...**, as shown in Figure 5.10.
 - b. In the pop-up file browser, select the following IPK path:
<Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip
\latticesemi.com_ahbl_to_axi4lite_bridge_1.0.0.0.ipk.
 - c. Click **OK**.
 - d. In the **IP License Agreement** window, click **Accept** to finish the IP installation, as shown in Figure 5.11.

- e. Repeat Steps a to d above to install the rest of the IPK listed below:
- <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi.com_ahbl_to_lmimi_bridge_1.1.0.0.ipk.
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi.com_ahbl_to_mem_bridge_1.0.0.0.ipk.
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_uvsi_to_usb_1.0.0.ipk
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_usb23_bus_intf_bridge_1.0.0.ipk
 - <Project Directory>\ RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_mipi_csi_dsi_1.0.0.02.ipk
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_ccm_1.3.1.01.ipk
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_color_space_converter_2.4.1.01.ipk
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_debayer_1.4.0.01.ipk
 - <Project Directory>\RD02306_mipi_csi_to_usb_uvc\misc\custom_ip\latticesemi_custom_chroma_resampler_1.0.0.ipk
- f. After completing the IPK installation, the IP mentioned above should be available under IP on Local, as shown in Figure 5.12.

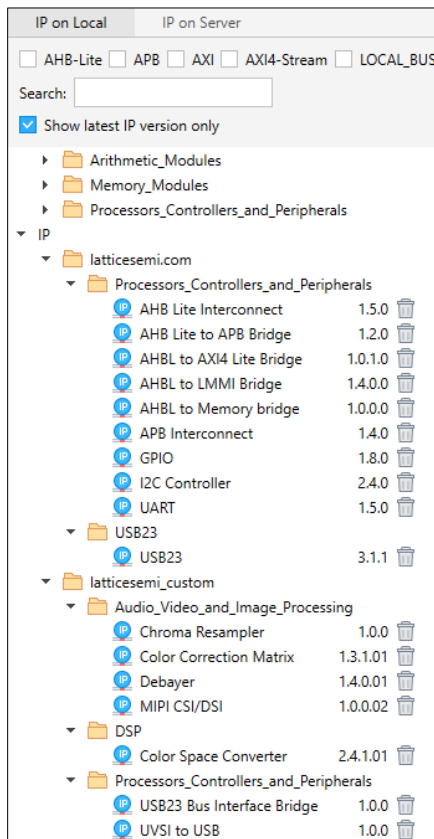


Figure 5.9. IP on Local Tab in IP Catalog

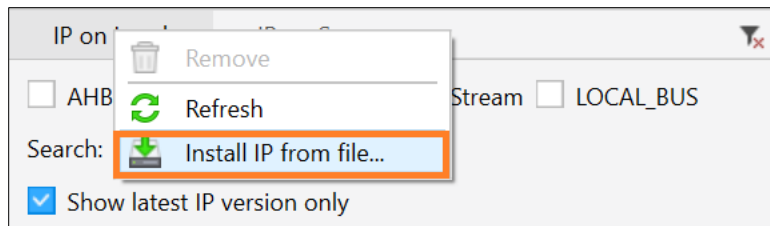


Figure 5.10. Install IP from File

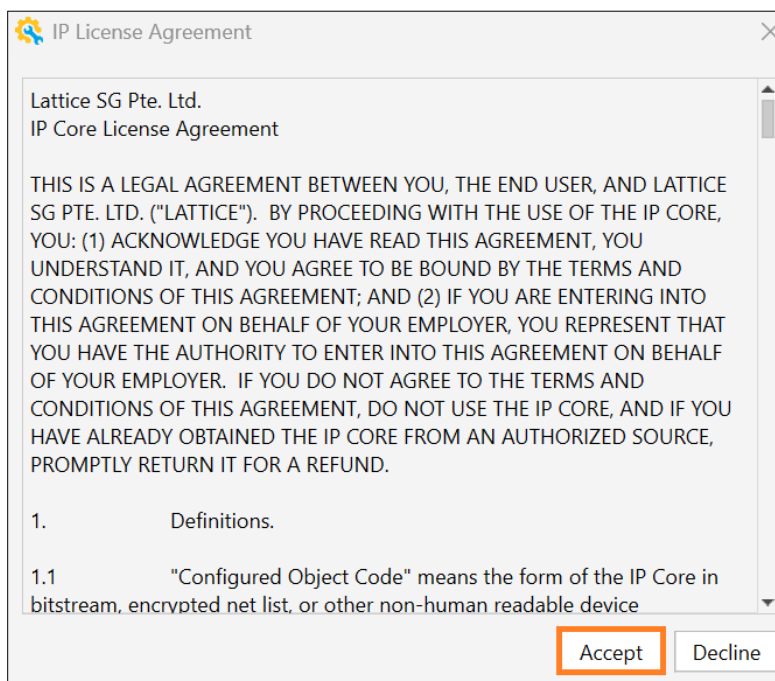


Figure 5.11. IP License Agreement

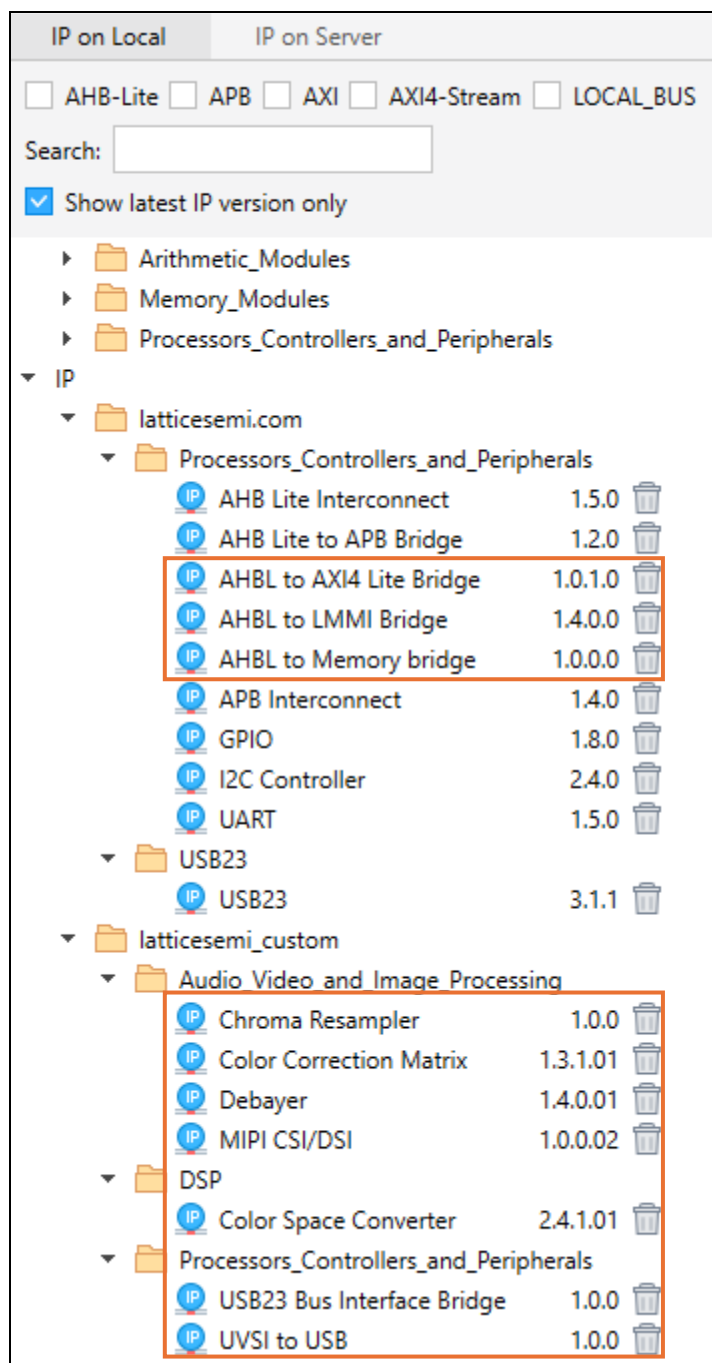


Figure 5.12. IPs Installed from Files

- In the Propel Builder design schematic, double-click the *sysmem0_inst* to open the system_memory Module/IP Block Wizard and ensure the Initialization File is mapped to `../propelsdk/riscv_mc/Debug/riscv_mc.mem` or the latest revision of the .mem file. Then, click on the **Generate** button, as shown in Figure 5.13.

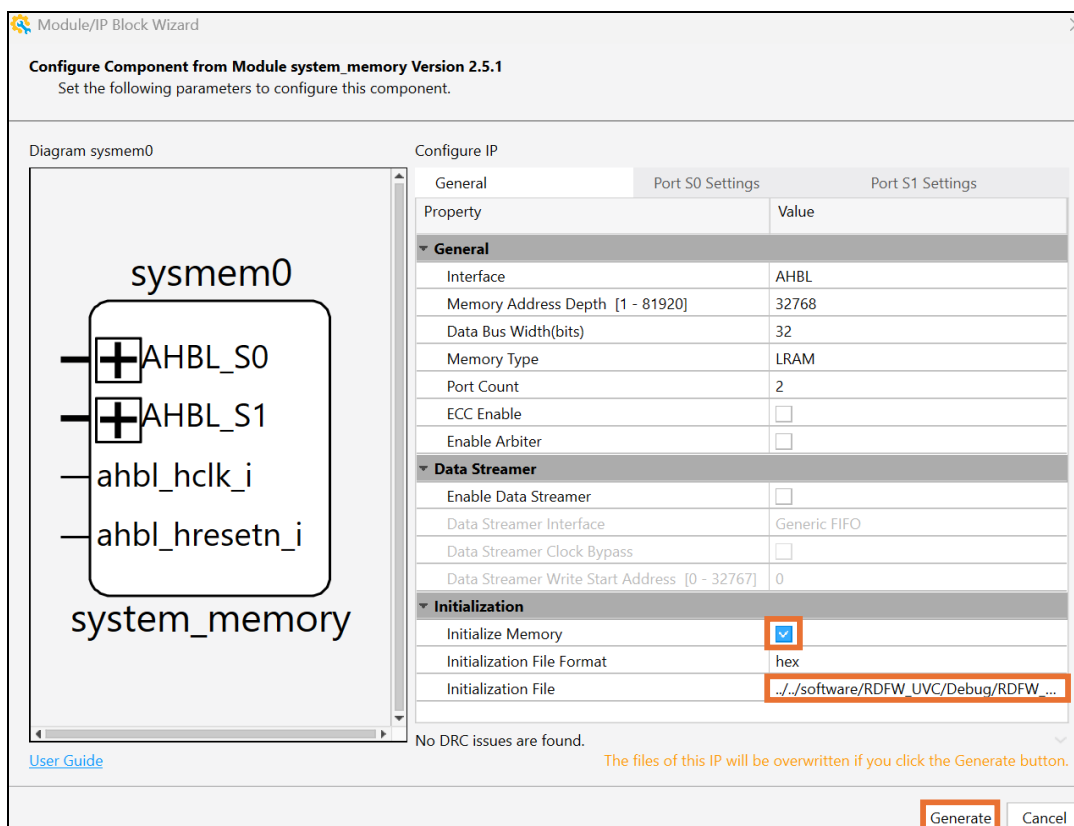


Figure 5.13. Mapping the System Memory Initialization Files

- Click the **Address** tab to review the base address bits assignment for the system and all peripherals within the Propel Builder Project, as shown in Figure 5.14.

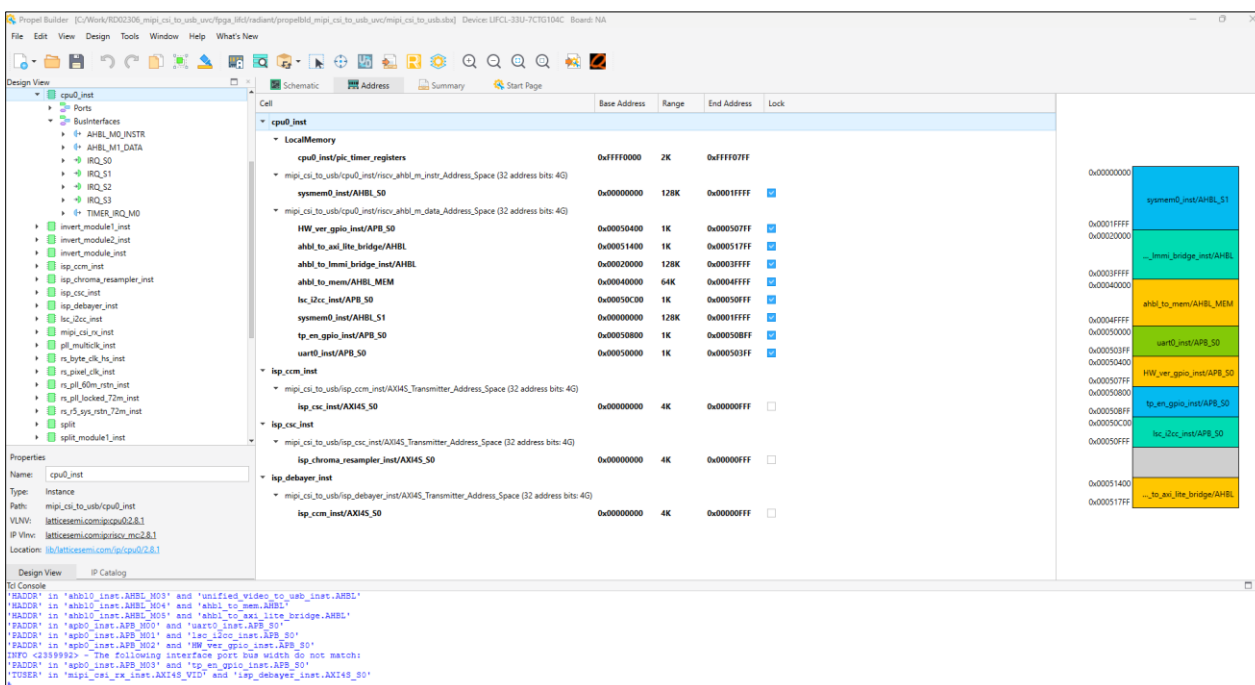


Figure 5.14. Base Address Assignment

Note: To reconfigure the IP, refer to the [Customizing the Reference Design](#) section for the available IP configuration options.

6. To validate the design, select **Design > Validate Design** or click the **Validate Design** icon (as shown in [Figure 5.15](#)). Ensure the TCL console shows no error, and only the expected INFO messages and WARNING appear.

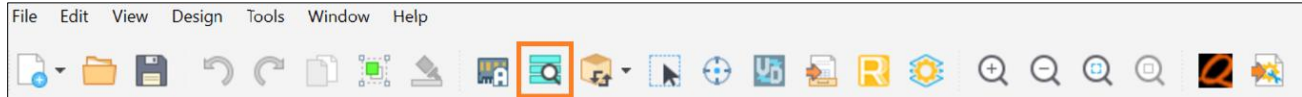


Figure 5.15. Validate the Design in Lattice Propel Builder

7. To generate the design, select **Design > Generate** or click the **Generate** icon (as shown in [Figure 5.16](#)). Ensure the TCL console shows no errors, and only the expected INFO messages and WARNING appear.

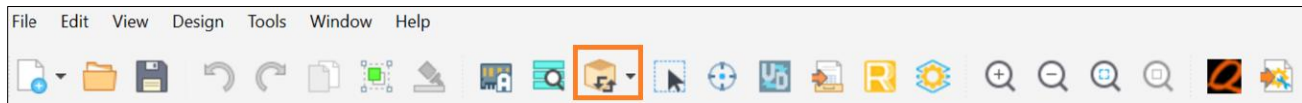


Figure 5.16. Generate in the Lattice Propel Design

```
Tcl Console

INFO <2359992> - (PGE FileExistence) No available driver for ahbl_to_axi4lite_bridge
INFO <2359992> - (PGE FileExistence) No available driver for ahbl_to_lmmi_bridge
INFO <2359992> - (PGE FileExistence) No available driver for ahbl_to_mem_bridge
%
```

Figure 5.17. TCL Console Output after Generating the Design

8. To synthesize the RTL files and generate the bitstream, click the Lattice Radiant icon in Lattice Propel Builder, as shown in [Figure 5.18](#).

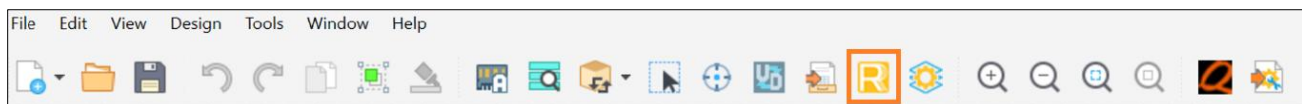


Figure 5.18. Lattice Radiant Icon in Lattice Propel Builder

9. The Lattice Radiant software window opens and shows this project ([Figure 5.19](#)).

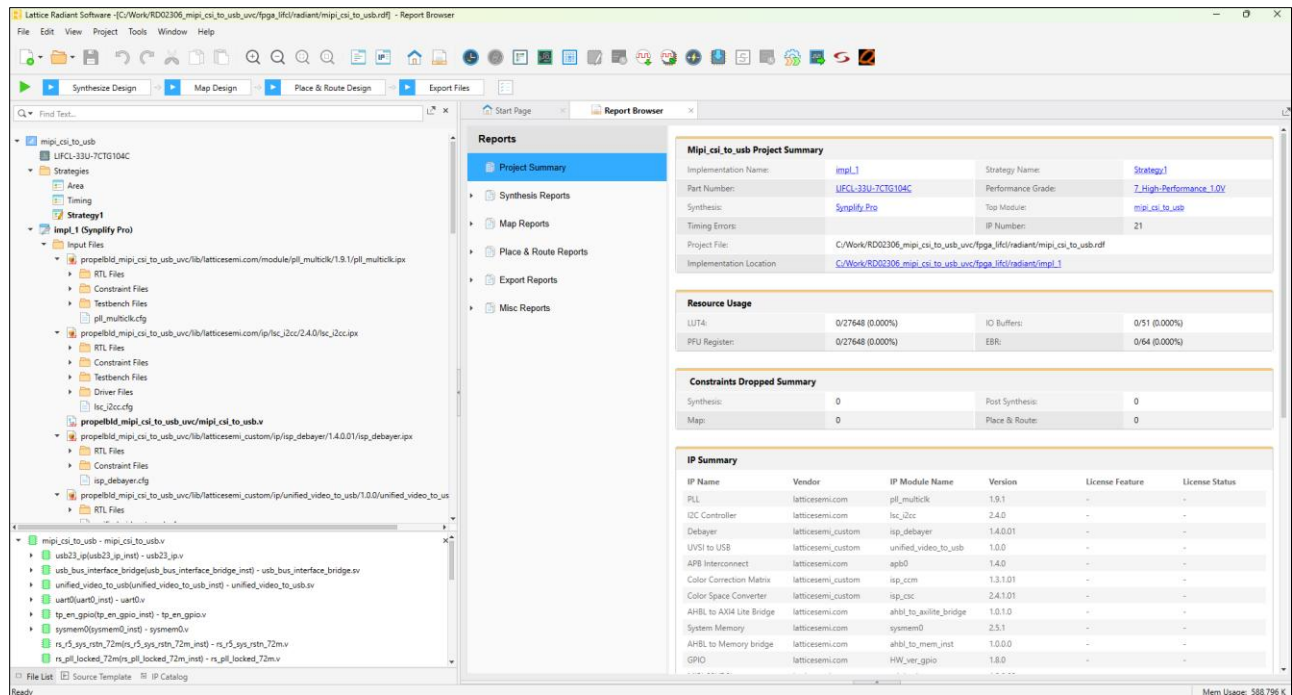


Figure 5.19. Lattice Radiant Software Window

10. Set the synthesis strategy (Figure 5.20) and the Place and Route design strategy (Figure 5.21).

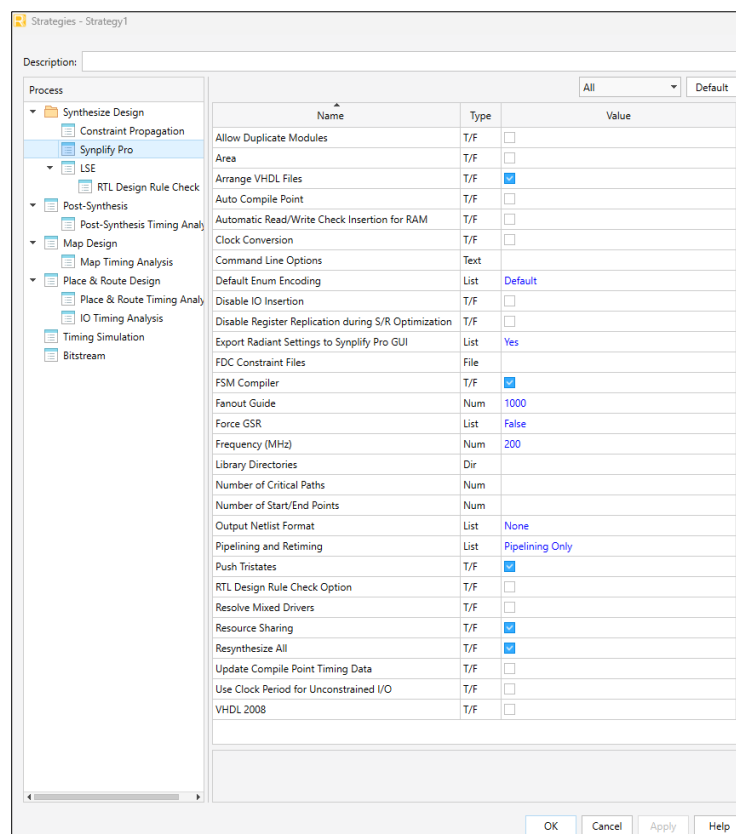


Figure 5.20. Synthesis Strategy Settings

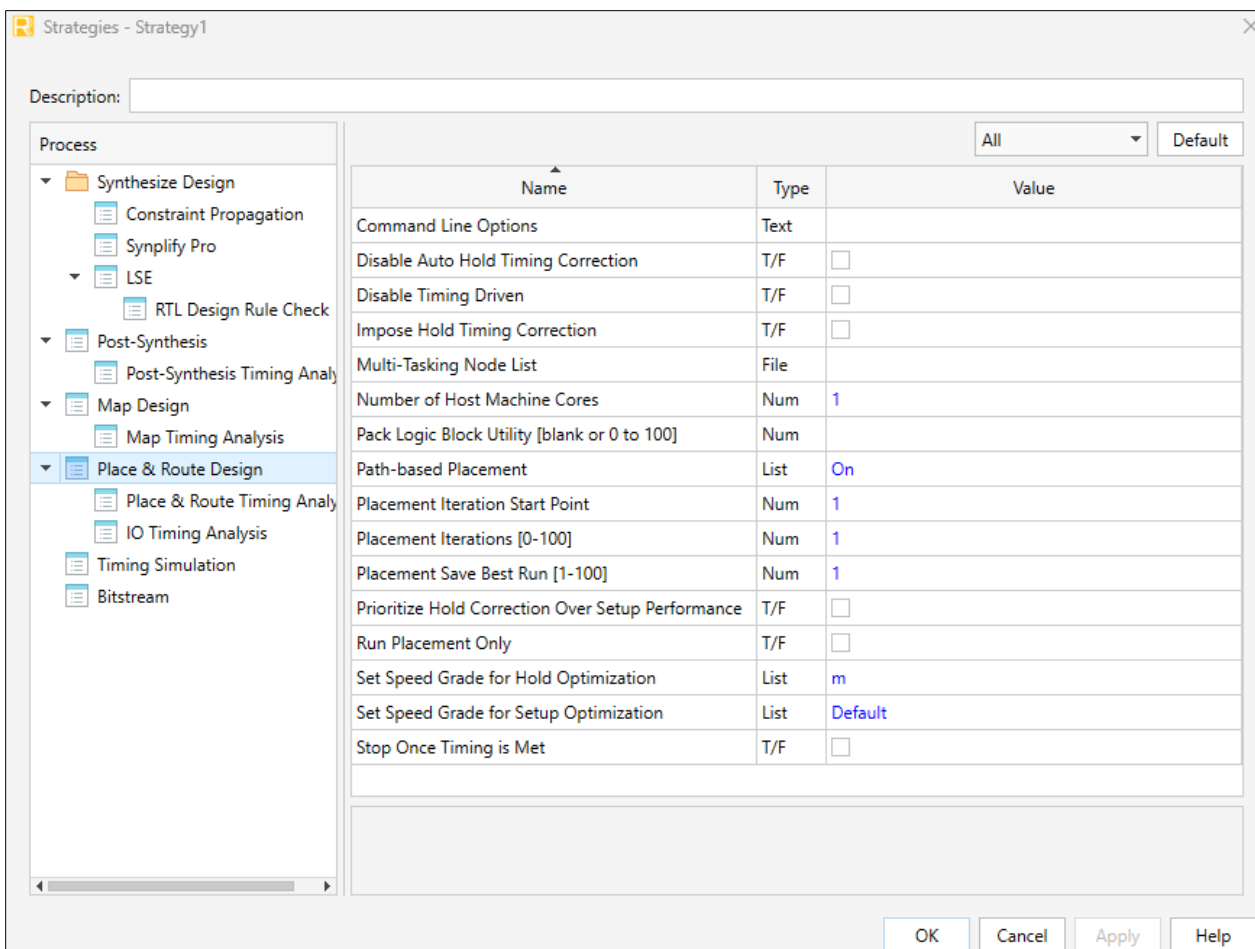


Figure 5.21. Place and Route Design Strategy Settings

- Click the **Run All** icon to generate the bitstream (Figure 5.22).

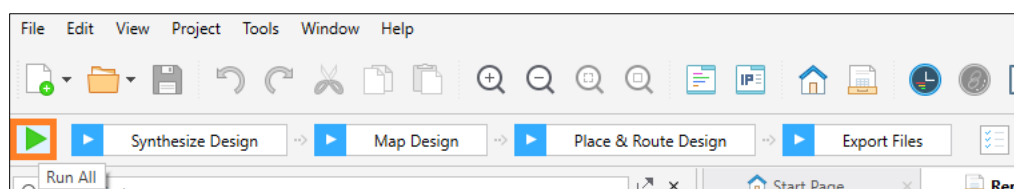


Figure 5.22. Generating Bitstream

- The compilation flow takes some time to complete. As shown in Figure 5.23, when the compilation is completed successfully, the `mipi_csi_to_usb_uvc_impl_1.bit` file is generated or updated in the project path:
`<Project Directory>/RD02306_mipi_csi_to_usb_uvc/fpga_lifcl/radiant/impl_1.`



Figure 5.23. Compilation Completed

Note: If timing violations occur or Place and Route failed during compilation or fail to generate bitstream, refer to the [Debugging](#) section for detailed debugging instructions.

6. Implementing the Reference Design on Board

This section describes how to run this reference design on the CrossLinkU-NX Evaluation Board.

6.1. Programming the FPGA

To program the FPGA, follow these steps:

1. Connect the Micro USB port (J2) of the CrossLinkU-NX Evaluation Board to your PC by using a Micro USB cable.
2. Create a new project using the Lattice Radiant Programmer software. In the **Getting Started** dialog box, enter the **Project Name** and **Location**, as shown in Figure 6.1.
3. Select **Create a new project from a scan**. The values are indicated in the **Cable**, **Port**, and **TCK Divider Setting (0-30x)** fields.
4. Click **OK**.

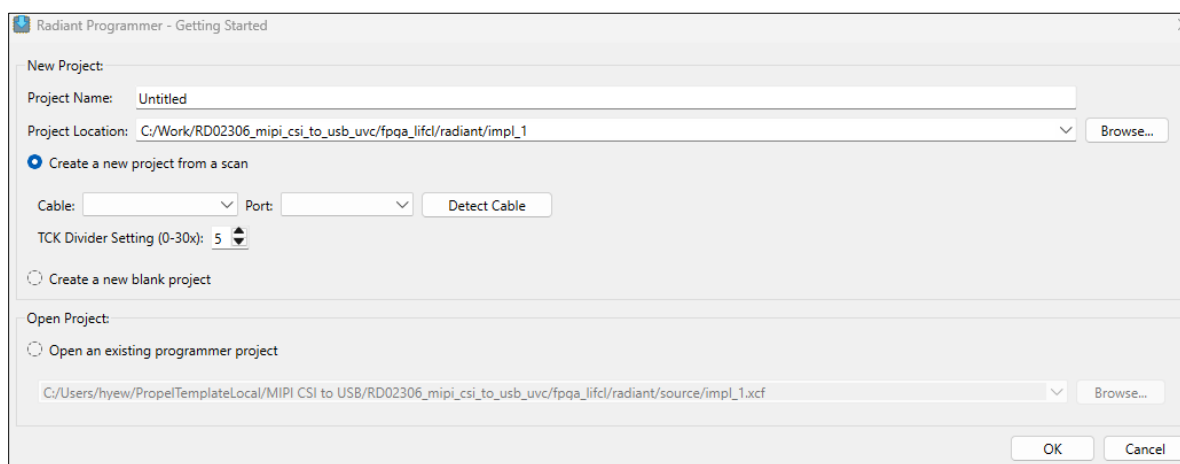


Figure 6.1. Creating a New Project from a Scan

5. The Radiant Programmer main interface opens, as shown in Figure 6.2.

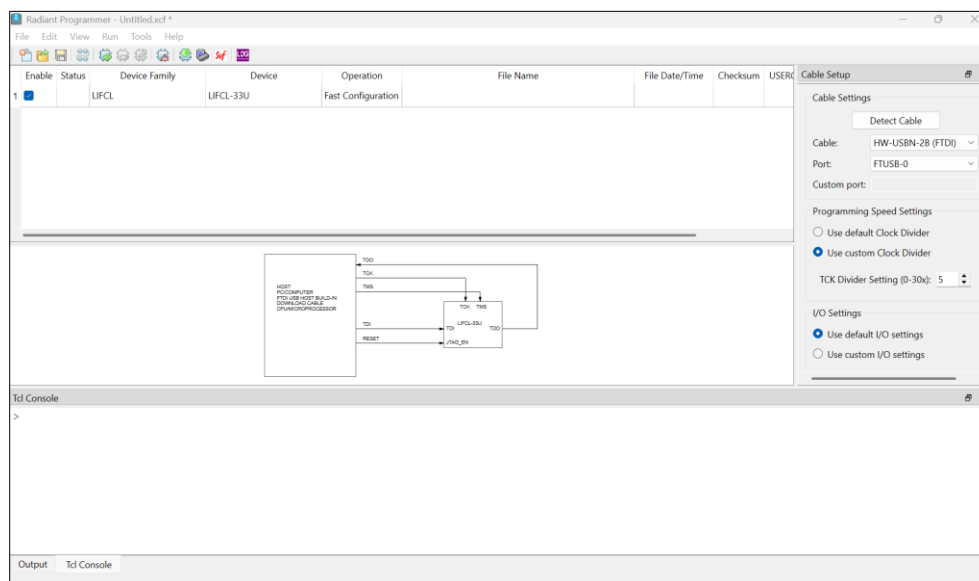


Figure 6.2. Lattice Radiant Programmer Window

6. If the Programmer settings do not match the settings for CrossLinkU-NX devices shown in Figure 6.3, select these settings manually from the drop-down menu.

Enable	Status	Device Family	Device	Operation	File Name	File Date/Time	Checksum	USERCODE
☒		LIFCL	LIFCL-33U	Fast Configuration				

Figure 6.3. CrossLinkU-NX Device Settings

To select the programming settings, follow these steps:

- Browse and select the Programming file, mipi_csi_to_usb_uvc_impl_1.bit.
- Click **OK**.
- Double-click **Operation** to open the **Device Properties** dialog box.
- Select the settings, as shown in Figure 6.4.

Note: If the flash device is a GigaDevice flash, then the SPI Flash options are GD25LQ128E and 8-pin SOP. If the flash device is a Winbond flash, then the SPI Flash options are W25Q512JV and 8-pad WSON.

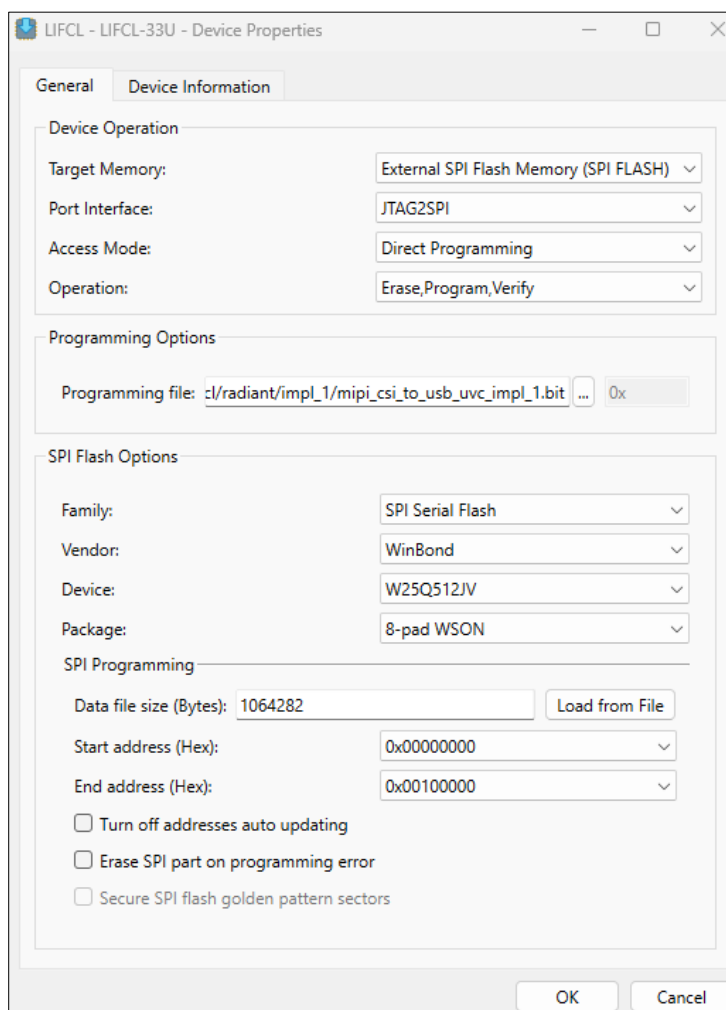


Figure 6.4. Device Properties Window for the CrossLinkU-NX SPI Flash Programming

7. Click **Programming** from the menu bar to start programming (Figure 6.5).

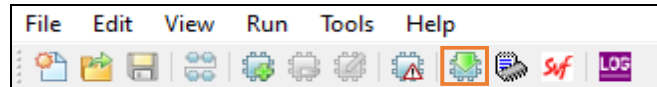


Figure 6.5. Programmer Menu Bar

When the FPGA programming is successful, the output console displays the *Operation: successful* message (Figure 6.6).

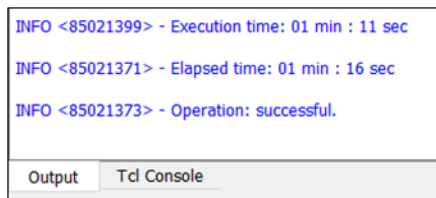


Figure 6.6. Programming Output Window

If the programming operation is unsuccessful, refer to the [Debugging](#) section.

6.2. Hardware Setup

Connect the Raspberry Pi Camera Module 2 to the CN1 connector on the CrossLinkU-NX Evaluation Board, which is the camera connector. Connect the USB Type C connector CN2 on the pre-programmed CrossLinkU-NX Evaluation Board to the PC using the USB type C to USB type C cable or the USB type C to USB type A 9-pin cable, as shown in Figure 6.7. Now the device is enumerated as Lattice USB23, which can be found in Windows Device Manager, as shown in Figure 6.8.

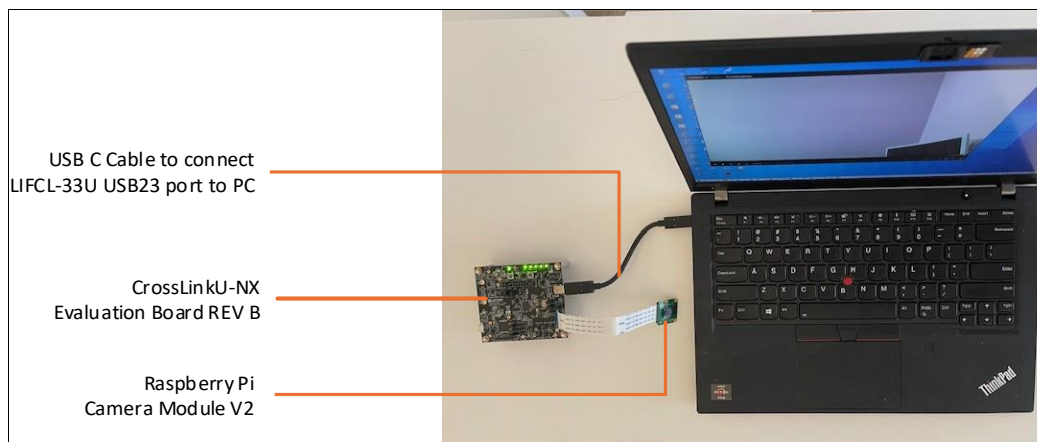


Figure 6.7. CrossLinkU-NX Evaluation Board Setup for Demonstration

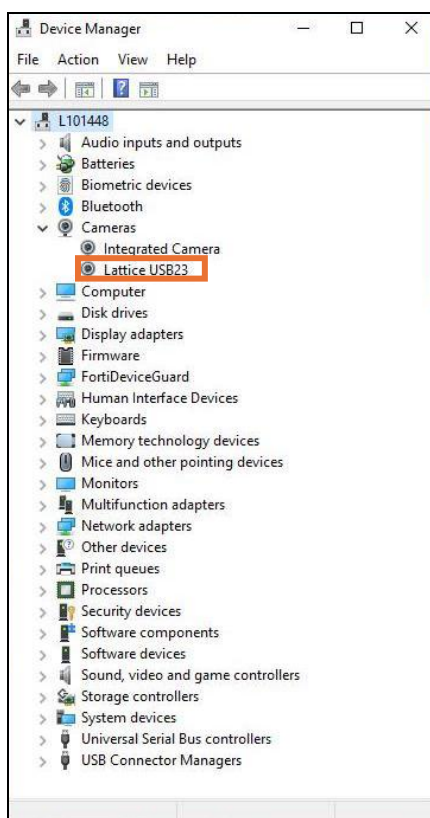


Figure 6.8. USB Enumeration

6.3. Demonstration

To demonstrate this reference design, follow these steps:

1. Prior to running this demonstration, ensure PotPlayer is installed. The version used in this reference design is version 1.7.
2. Open the PotPlayer application.
3. To open video, either hit the shortcut Ctrl+J or Click **PotPlayer > Open > Open Webcam/Other Device**, as shown in Figure 6.9.

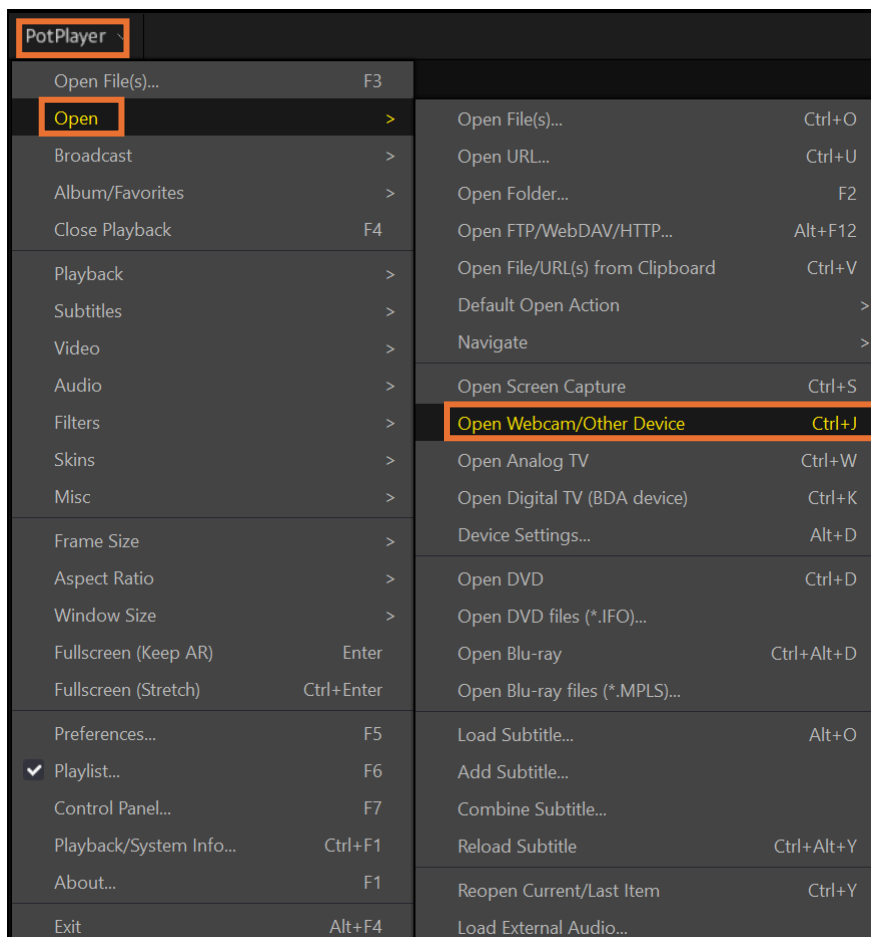


Figure 6.9. Open Webcam or Other Devices in PotPlayer

4. The video of the camera sensor can be seen on the PotPlayer screen, as shown in Figure 6.10.

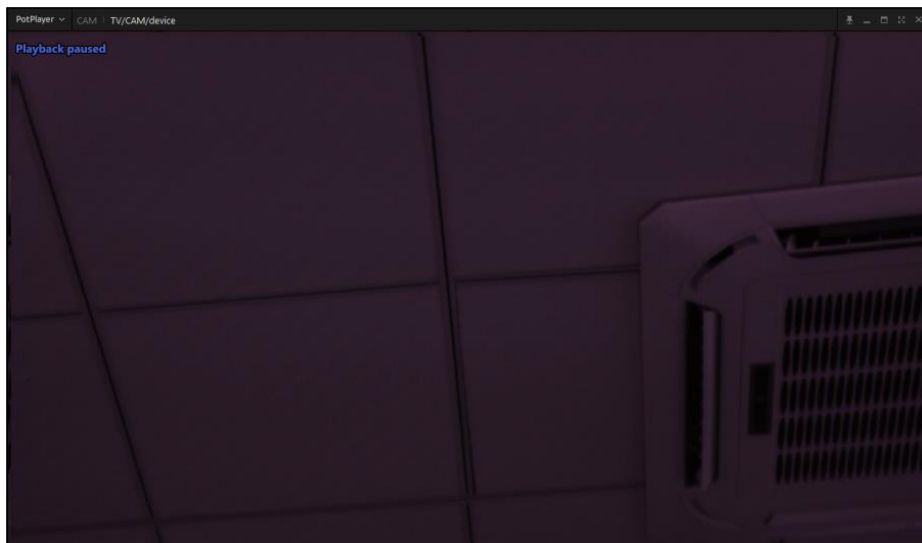


Figure 6.10. Video Output on PotPlayer

- To change the Video Renderer option in PotPlayer, right-click on the video and select **Video > Video Renderer > Built in Direct3D 11 Video Renderer**, as shown in Figure 6.11.

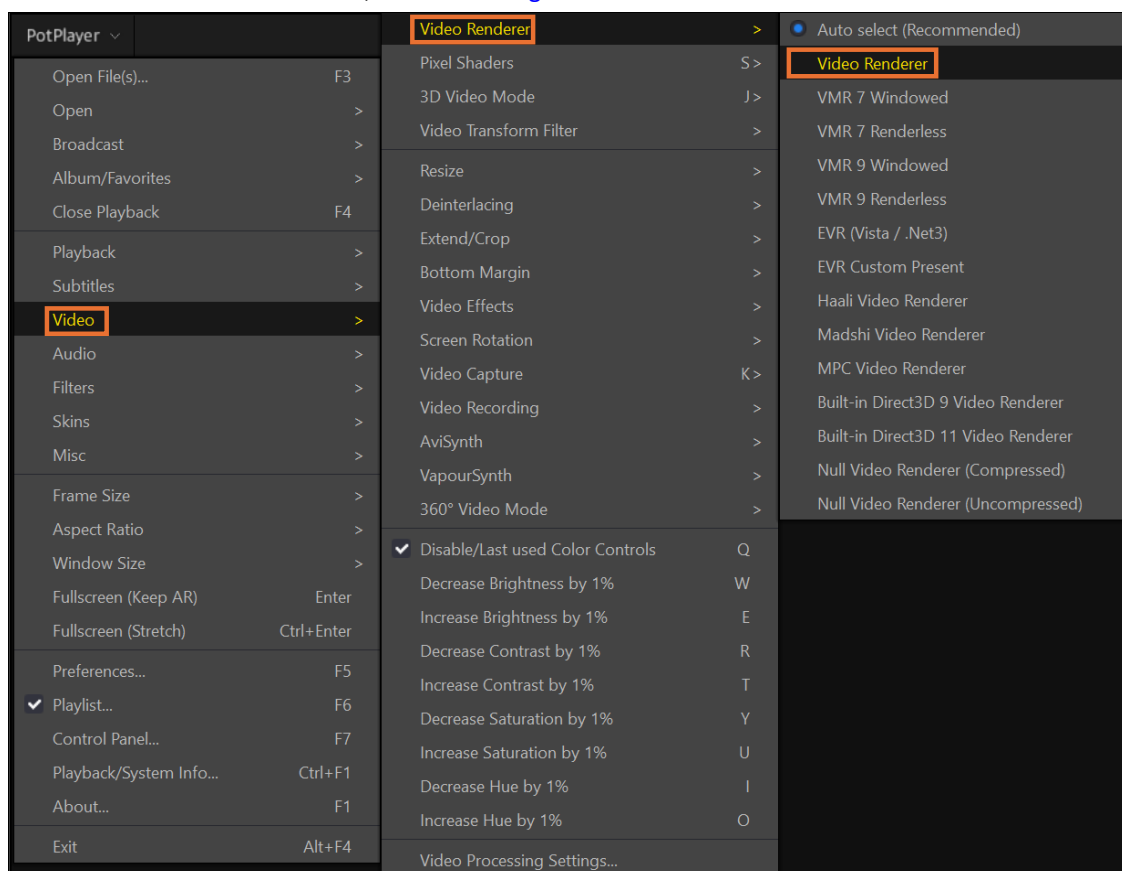


Figure 6.11. Video Renderer Option

- To view the frame per second (FPS), either hit the shortcut Ctrl + F1 or click **PotPlayer > Playback/System Info ...**, as shown in Figure 6.12. An example of the video information is shown in Figure 6.13.

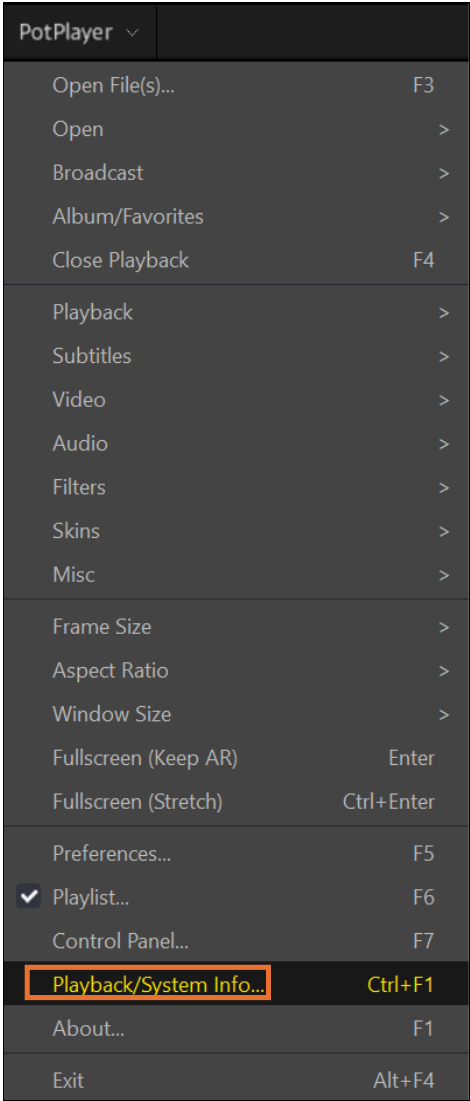


Figure 6.12. Playback/System Information

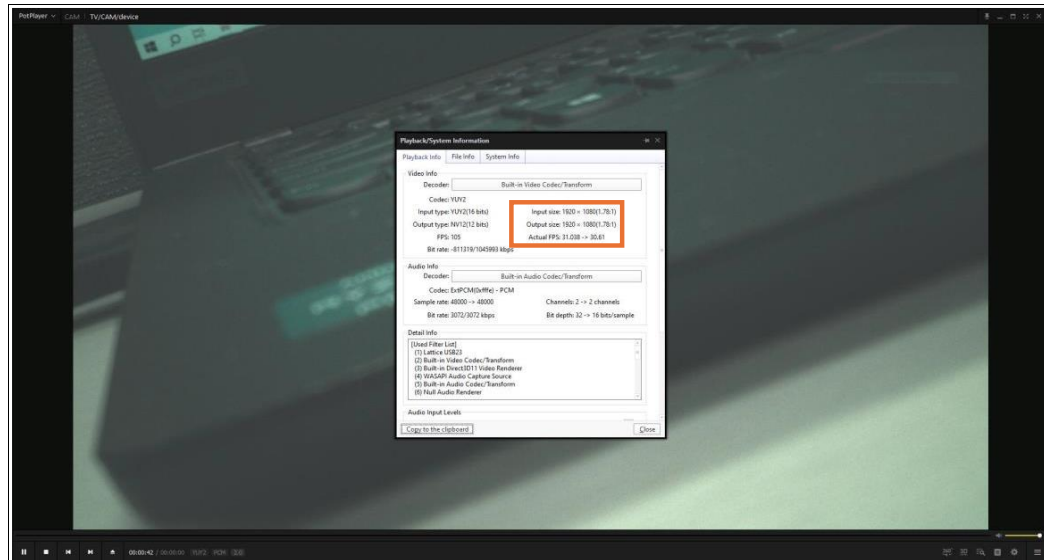


Figure 6.13. Example of Video Information

7. Customizing the Reference Design

This section describes the steps required to configure each IP and software for various demonstration setups, starting from the default design configuration. It also provides a way to enable or disable the YUV422 test pattern generator.

While the default IP configuration can support all resolutions and frame rates, IP can be reconfigured to demonstrate lower pixel per clock setup resulting in reduced resource utilization. The IMX219 sensor is configured to operate at 592 Mbps link rate with 2 lanes configuration. Refer to [Table 7.1](#) for the IP configuration setting according to the desired setup.

Table 7.1. IP Configuration for Supported Resolutions

Resolution	D-PHY Link Rate (Mbps)	D-PHY Lane Count ^{1,2}	Pixels Per Clock Required ^{3,4} (Clocking at 120 MHz)
1080p	592	2	1 or 2
720p	592	2	1 or 2

Notes:

1. IMX219 does not support 4 lanes configuration.
2. Driver does not support 1 lane configuration.
3. Pixels Per Clock of 4 is not supported due to high device resource requirement.
4. Clock frequency required is clocking ratio multiplied by byte clock frequency, whereby clocking ratio is calculated using following formula:

$$(\text{NUM_LANES} * \text{PHY_DATA_WIDTH}) / (\text{PPC} * \text{BPP})$$
 whereby in this reference design NUM_LANES is 2, PHY_DATA_WIDTH is 8, PPC is 1 or 2 or 4, BPP is 10 (RAW10) with minimum ratio being 1, and byte clock frequency is 592 Mbps / 8 = 74 MHz.
 - NUM_LANES 2, PPC 1: $(2*8) / (1*10) = 1.6x$ Clocking Ratio; Clocking Required: $1.6 \times 74 \text{ MHz} = 118.4 \text{ MHz}$.
 - NUM_LANES 2, PPC 2: $(2*8) / (2*10) = 0.8x$ Clocking Ratio; Clocking Required: $1 \times 74 \text{ MHz} = 74 \text{ MHz}$.
 - NUM_LANES 2, PPC 4: $(2*8) / (4*10) = 0.4x$ Clocking Ratio; Clocking Required: $1 \times 74 \text{ MHz} = 74 \text{ MHz}$.
 For more information, refer to <Project Directory>/latticesemi_custom_ip_mipi_csi_dsi_1.0.0.02/doc/IP_Design_Notes.pdf for IP user guide.

7.1. Change the Camera Resolution

This section explains how to change the camera video resolution, which can be configured through PotPlayer configuration.

7.1.1. Selecting the 1080p Output Resolution

1. Open the PotPlayer application.
2. To configure output resolution, either hit the shortcut Alt+D or Click **PotPlayer > Open > Device Settings...**, as shown in [Figure 7.1](#).

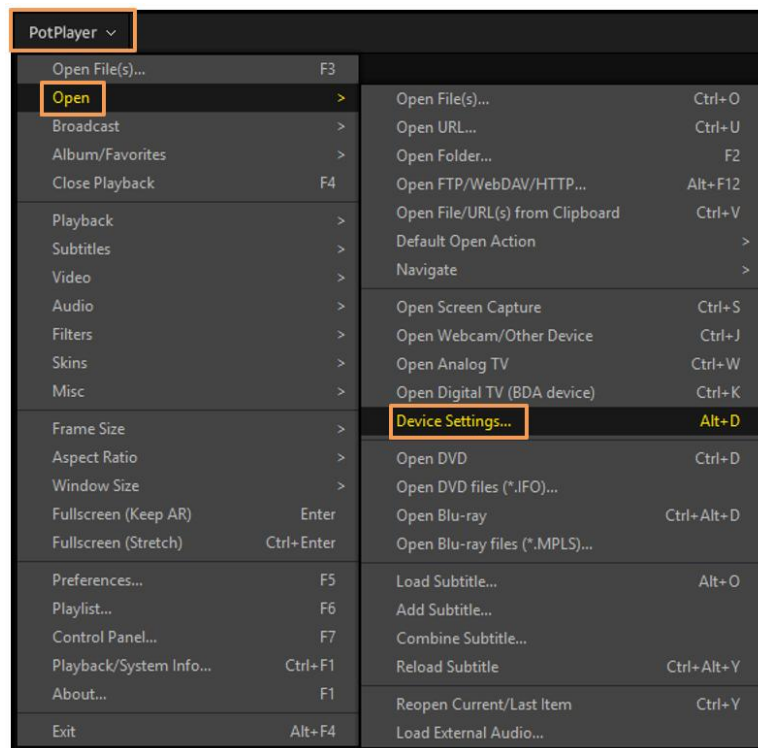


Figure 7.1. Device Settings in PotPlayer

3. In Format dropdown menu as shown in [Figure 7.2](#), select the YUV2 1920x1080 Option. Then click **OK**.

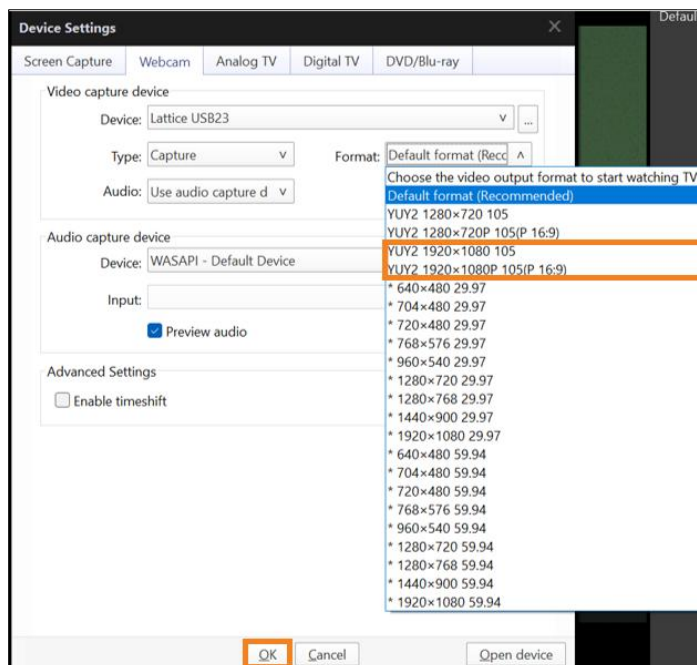


Figure 7.2. 1080p Selection in PotPlayer

4. Open video per normal settings, as described in the [Demonstration](#) section.

7.1.2. Selecting the 720p Output Resolution

1. Open the PotPlayer application.
2. To configure output resolution, either hit the shortcut Alt+D or Click **PotPlayer > Open > Device Settings...**, as shown in Figure 7.3.

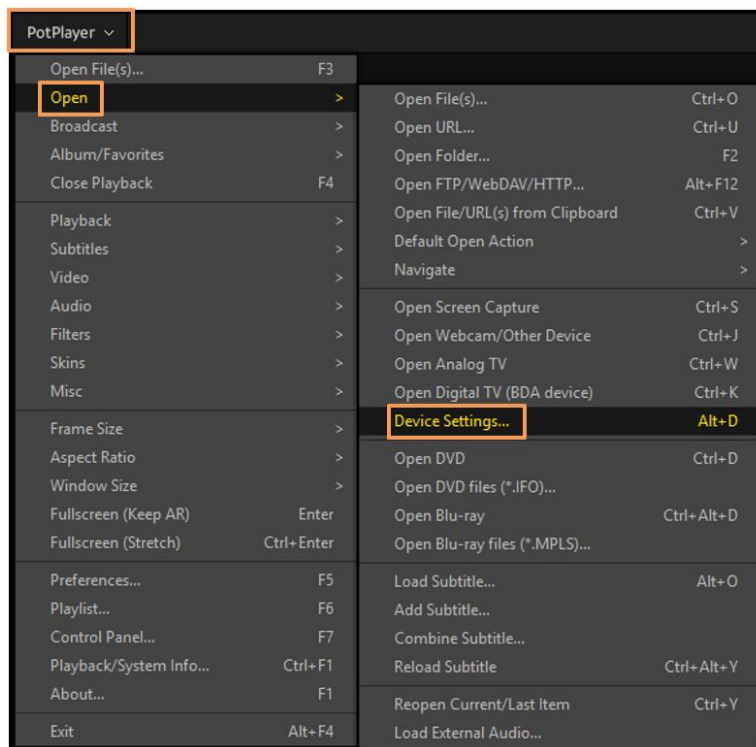


Figure 7.3. Device Settings in PotPlayer

3. In the Format drop-down menu shown in Figure 7.4, select the **YUV2 1920x1080** Option. Then click **OK**.

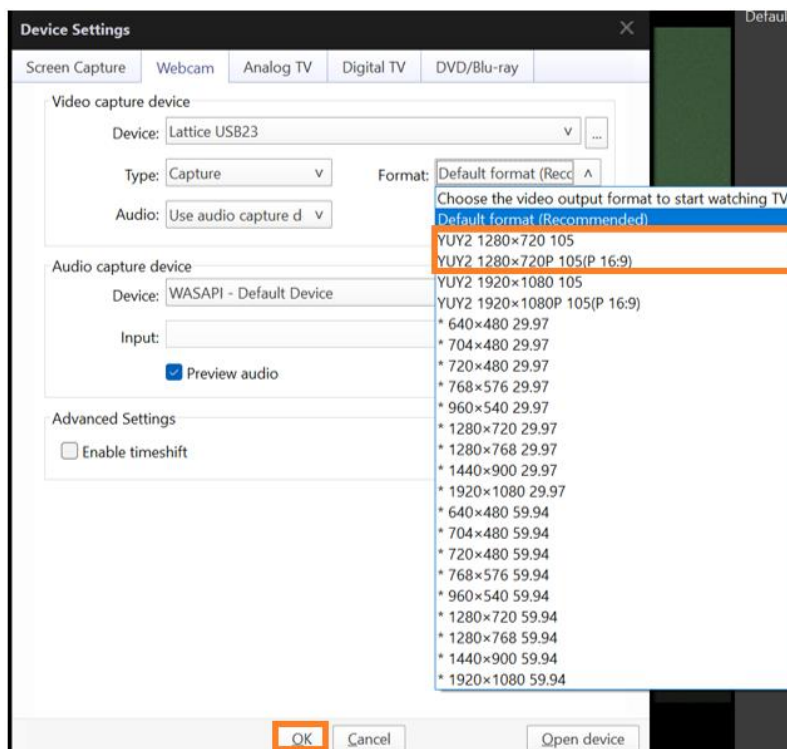


Figure 7.4. 720p Selection in PotPlayer

4. Open video per normal settings, as described in the [Demonstration](#) section.

7.2. Changing IP Configuration to 1 PPC

This section explains how to change the IP configurations to 1 PPC.

7.2.1. Configuring the MIPI RX IP

To configure the MIPI RX IP, follow these steps:

1. Double-click the `mipi_csi_rx_inst` block to open the Module/IP Block Wizard.
2. Under the **Byte-to-Pixel** setting, set the **Number of Pixel per Clock (PPC)** to 1 ([Figure 7.5](#)).

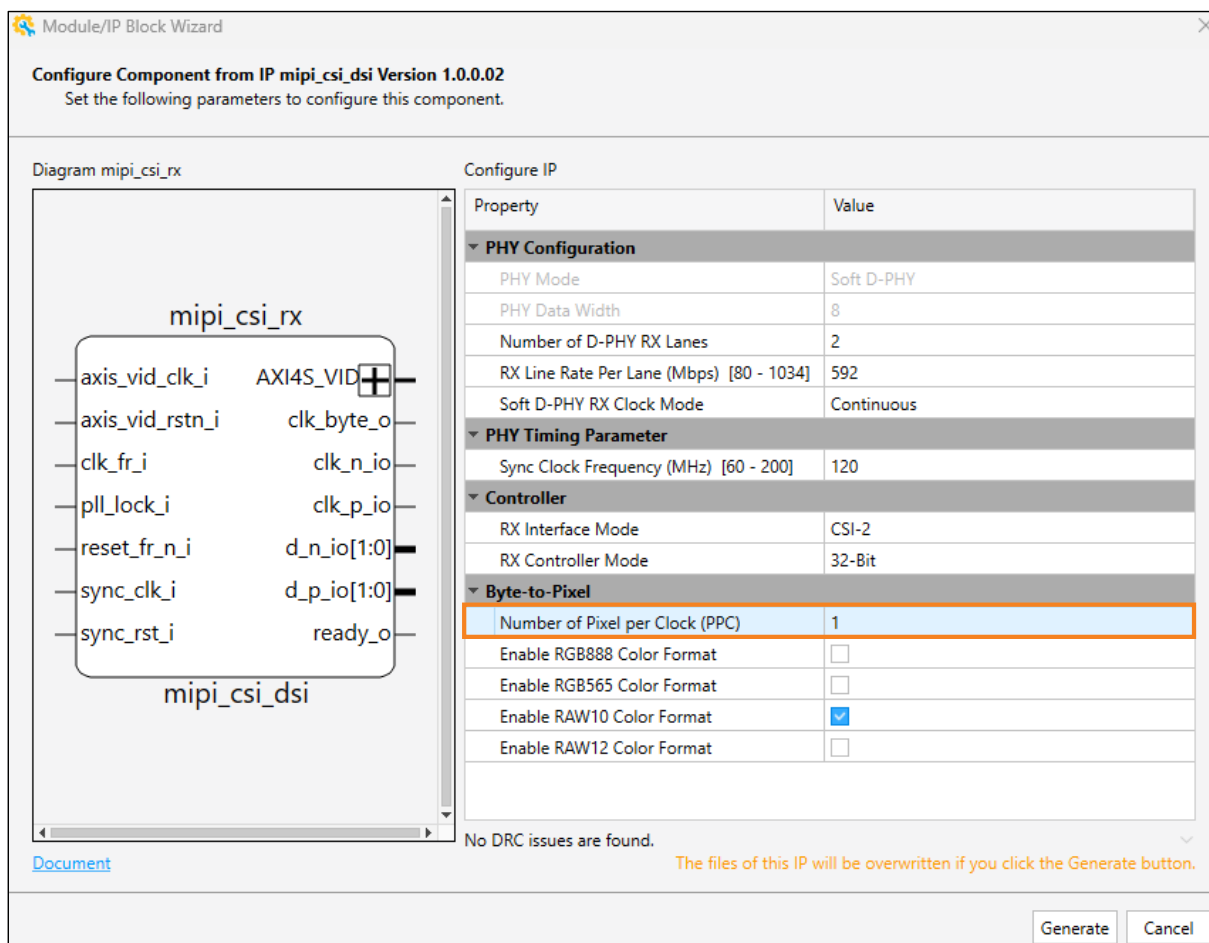


Figure 7.5. MIPI RX IP Configuration

3. Click **Generate**.
4. After the generation is completed, click **Finish** in the **Check Generated Result** window (Figure 7.6).

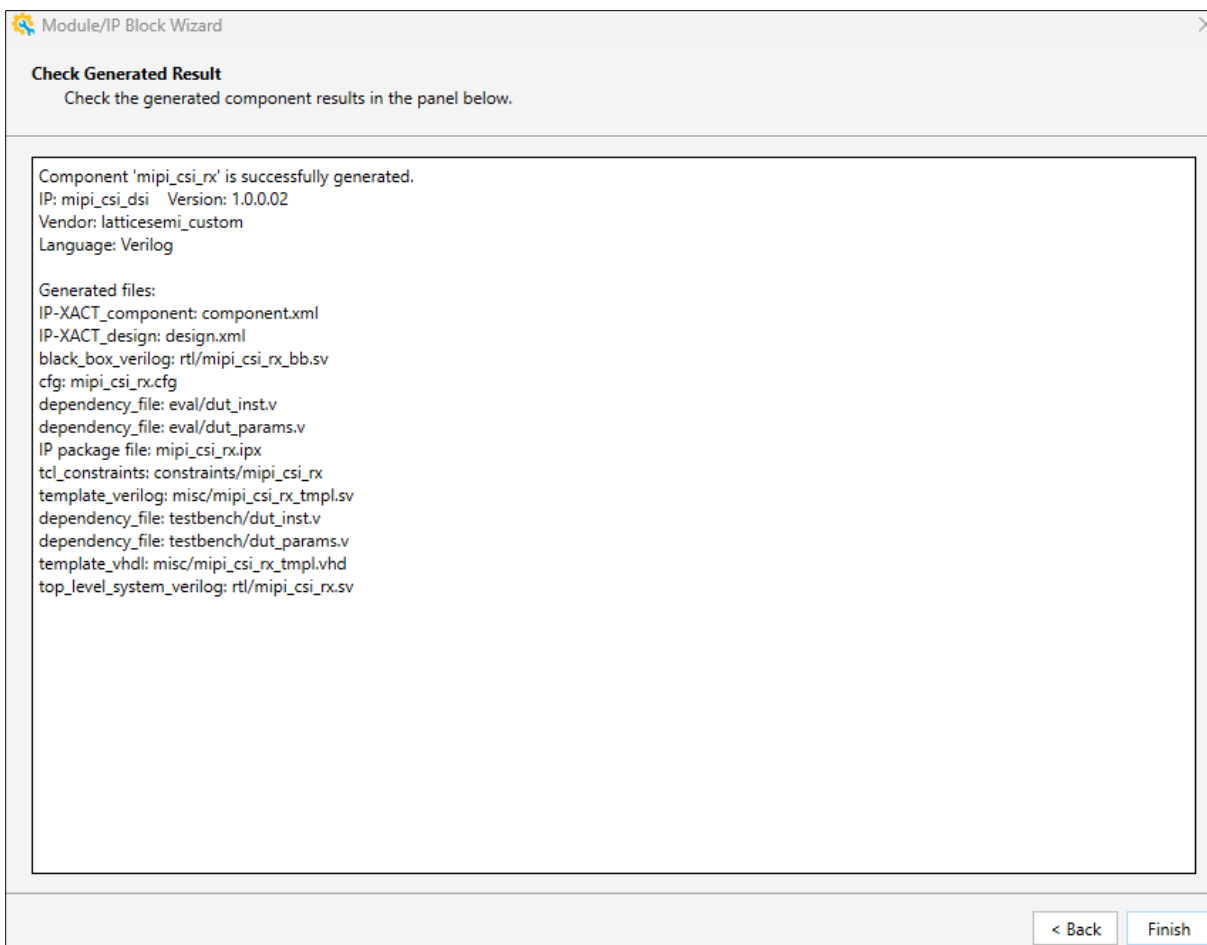


Figure 7.6. MIPI RX IP Check Generated Result Window

7.2.2. Configuring the Debayer IP

To configure the Debayer IP, follow these steps:

1. Double-click the `isp_debayer_inst` block to open the Module/IP Block Wizard.
2. Under the **DEBAYER** setting, set the **Pixels per clock** to 1 (Figure 7.7).

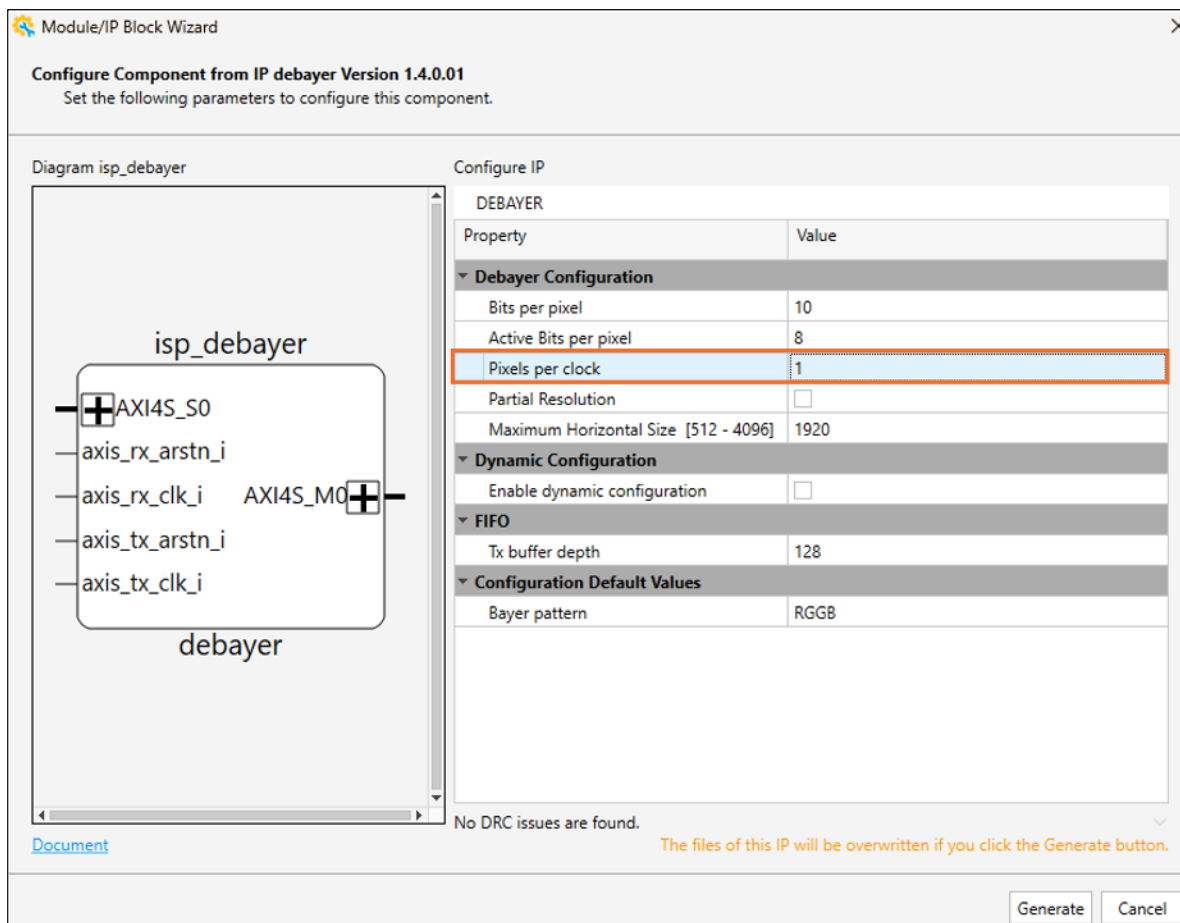


Figure 7.7. Debayer Configuration

3. Click **Generate**.
4. After the generation is completed, click **Finish** in the **Check Generated Result** window (Figure 7.8).

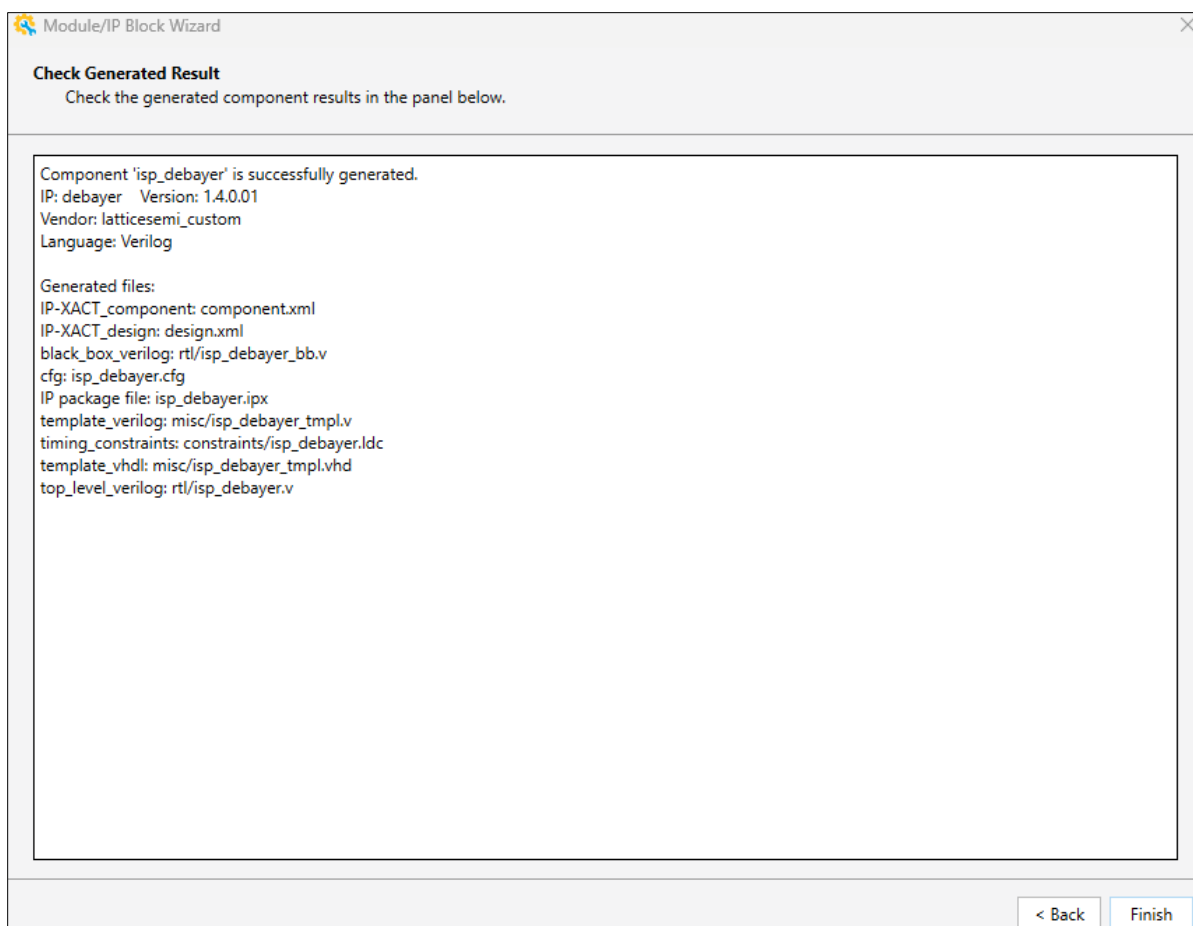


Figure 7.8. Debayer IP Check Generated Result Window

7.2.3. Configuring the Color Correction Matrix IP

To configure the Color Correction Matrix IP, follow these steps:

1. Double-click the isp_ccm_inst block to open the Module/IP Block Wizard.
2. Under the **CCM** setting, set the **Pixels per clock** to 1 (Figure 7.9).

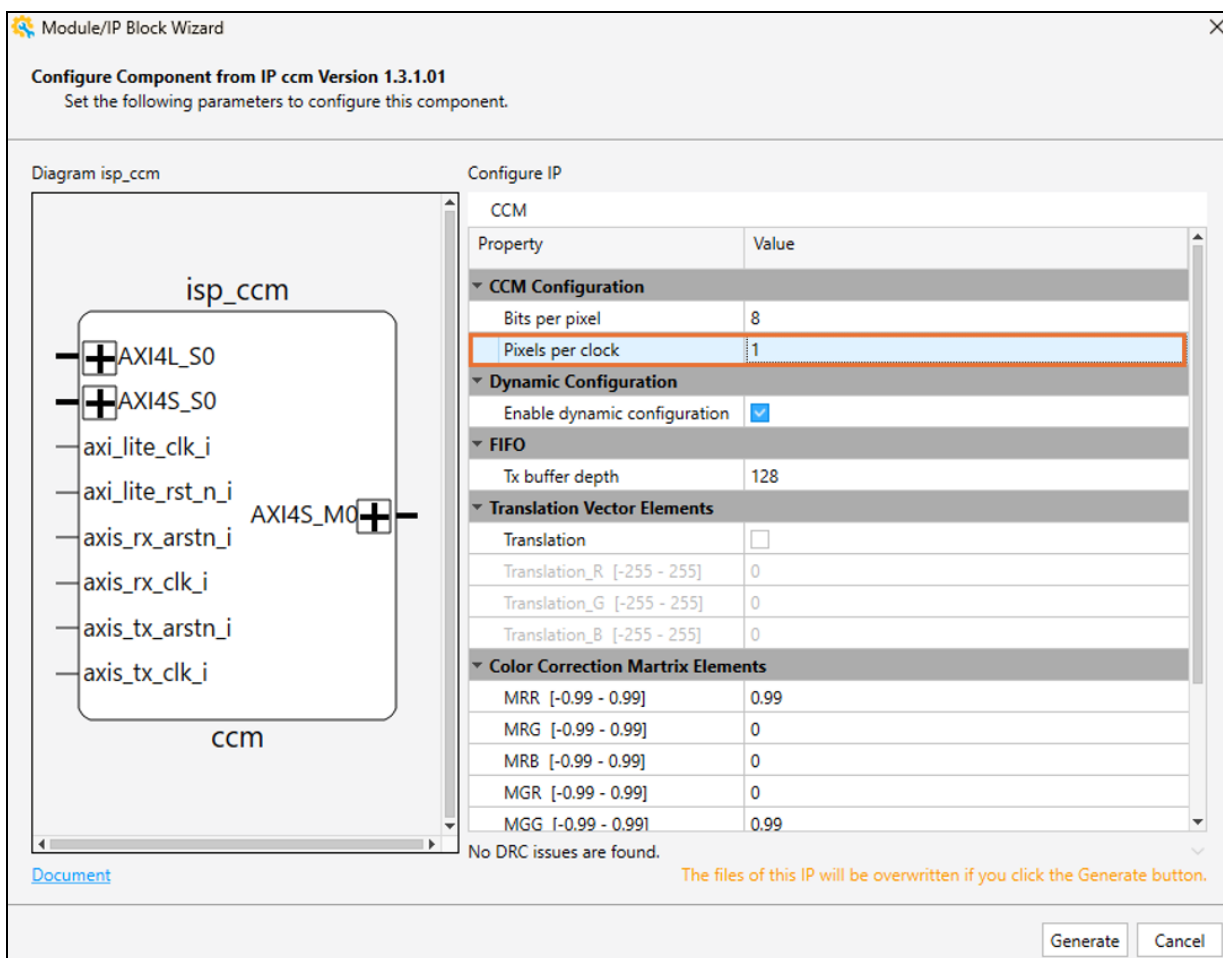


Figure 7.9. Color Correction Matrix Configuration

3. Click **Generate**.
4. After the generation is completed, click **Finish** in the **Check Generated Result** window (Figure 7.10).

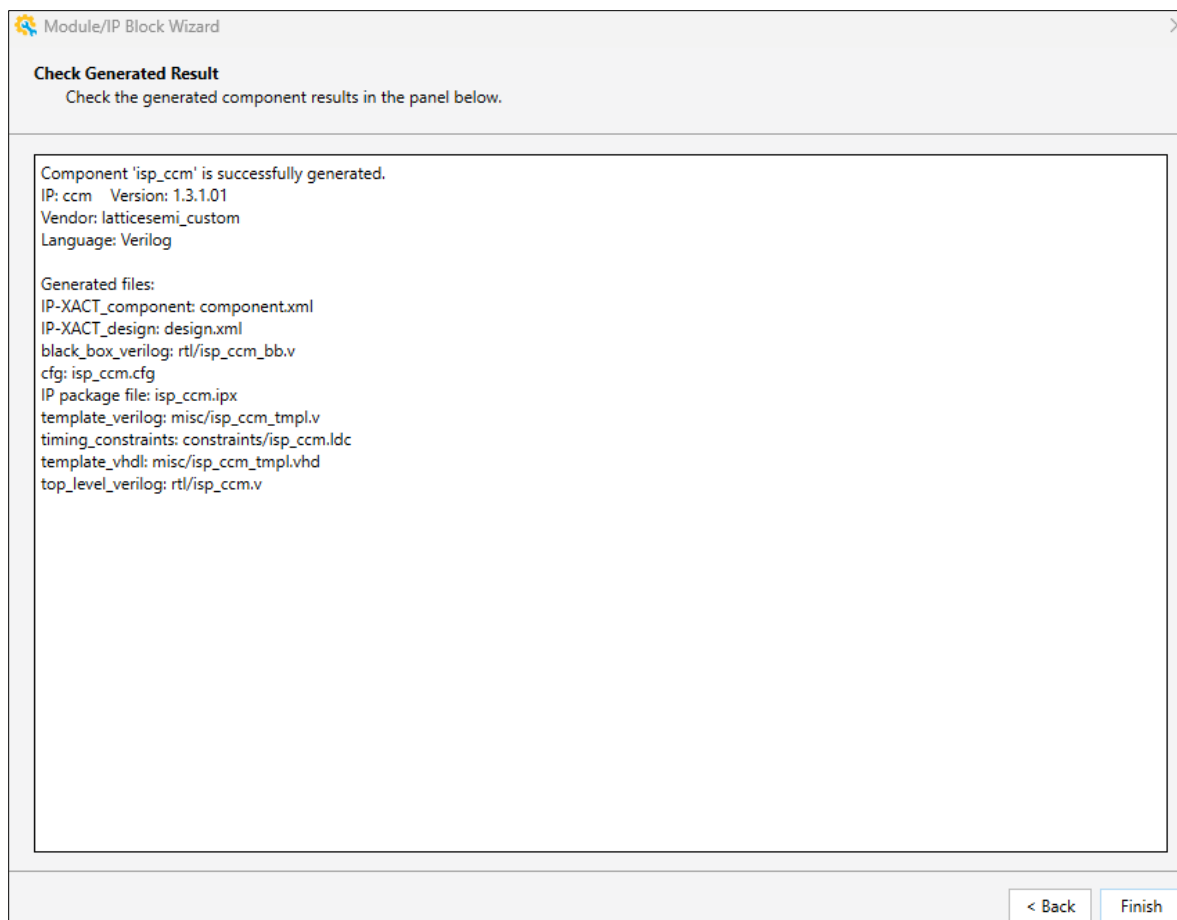


Figure 7.10. Color Correction Matrix IP Check Generated Result Window

7.2.4. Configuring the Color Space Converter IP

To configure the Color Space Converter IP, follow these steps:

1. Double-click the `isp_csc_inst` block to open the Module/IP Block Wizard.
2. Under the **Input setting**, set the **Pixels per clock** to 1 (Figure 7.11).

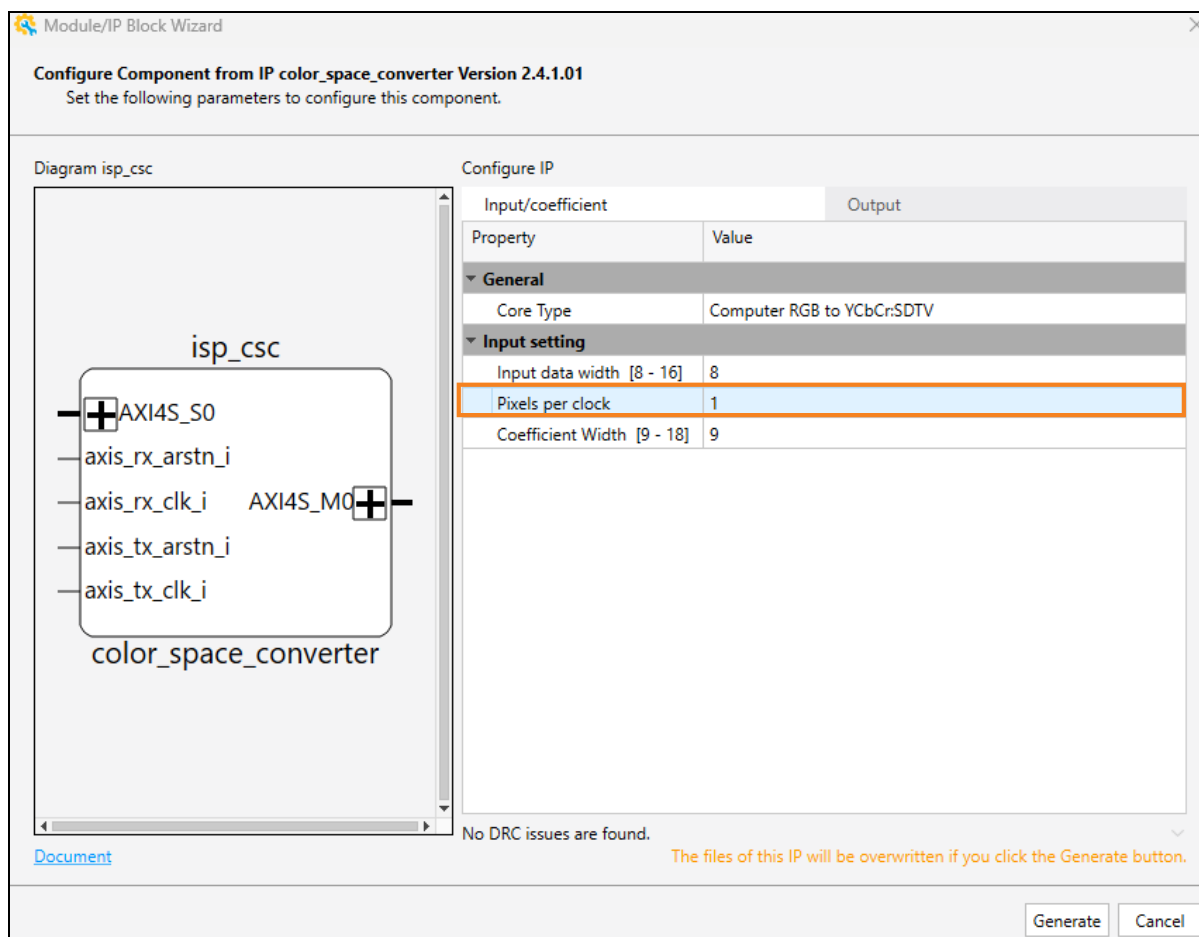


Figure 7.11. Color Space Converter Configuration

3. Click **Generate**.
4. After the generation is completed, click **Finish** in the **Check Generated Result** window (Figure 7.12).

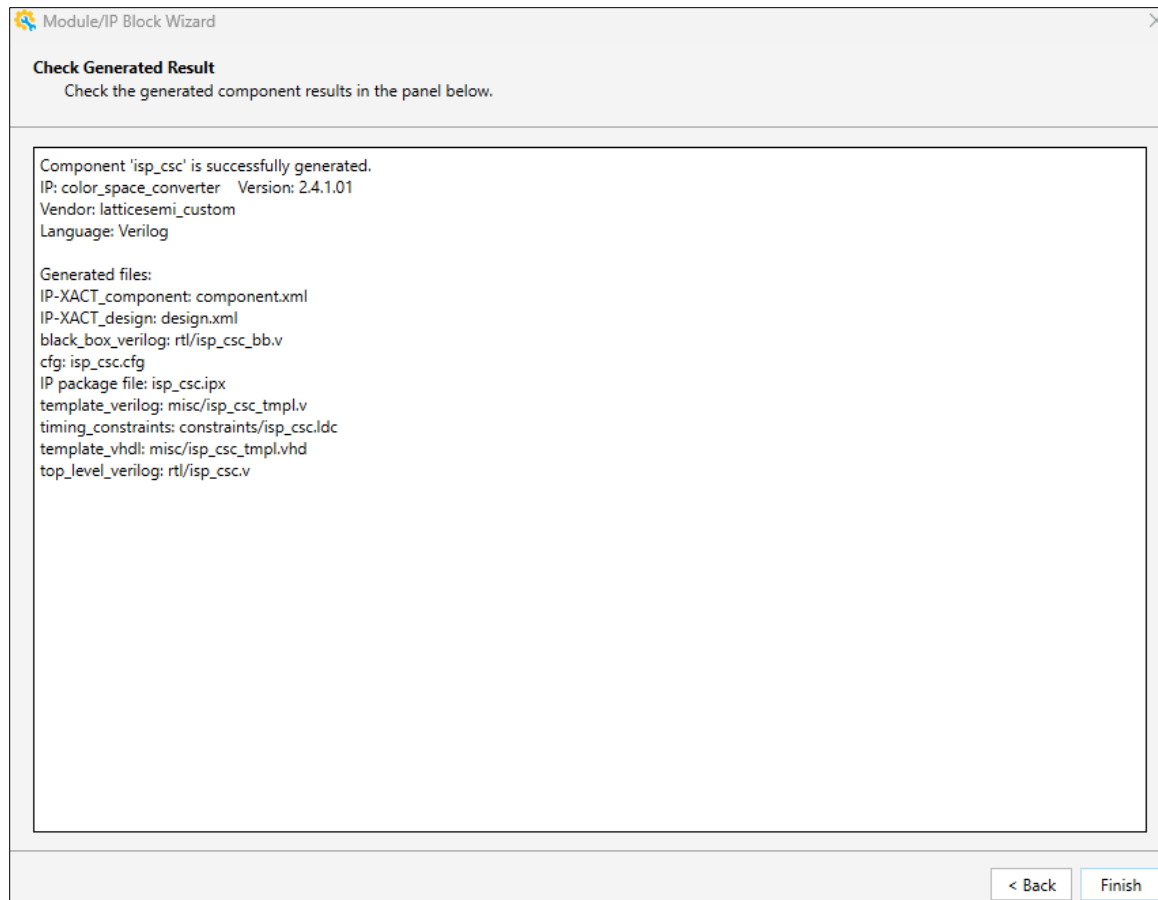


Figure 7.12. Color Space Converter IP Check Generated Result Window

7.2.5. Configuring the Chroma Resampler IP

To configure the Chroma Resampler IP, follow these steps:

1. Double-click the `isp_chroma_resampler_inst` block to open the Module/IP Block Wizard.
2. Under the **Input setting**, set the **Pixels per clock** to 1 (Figure 7.13).

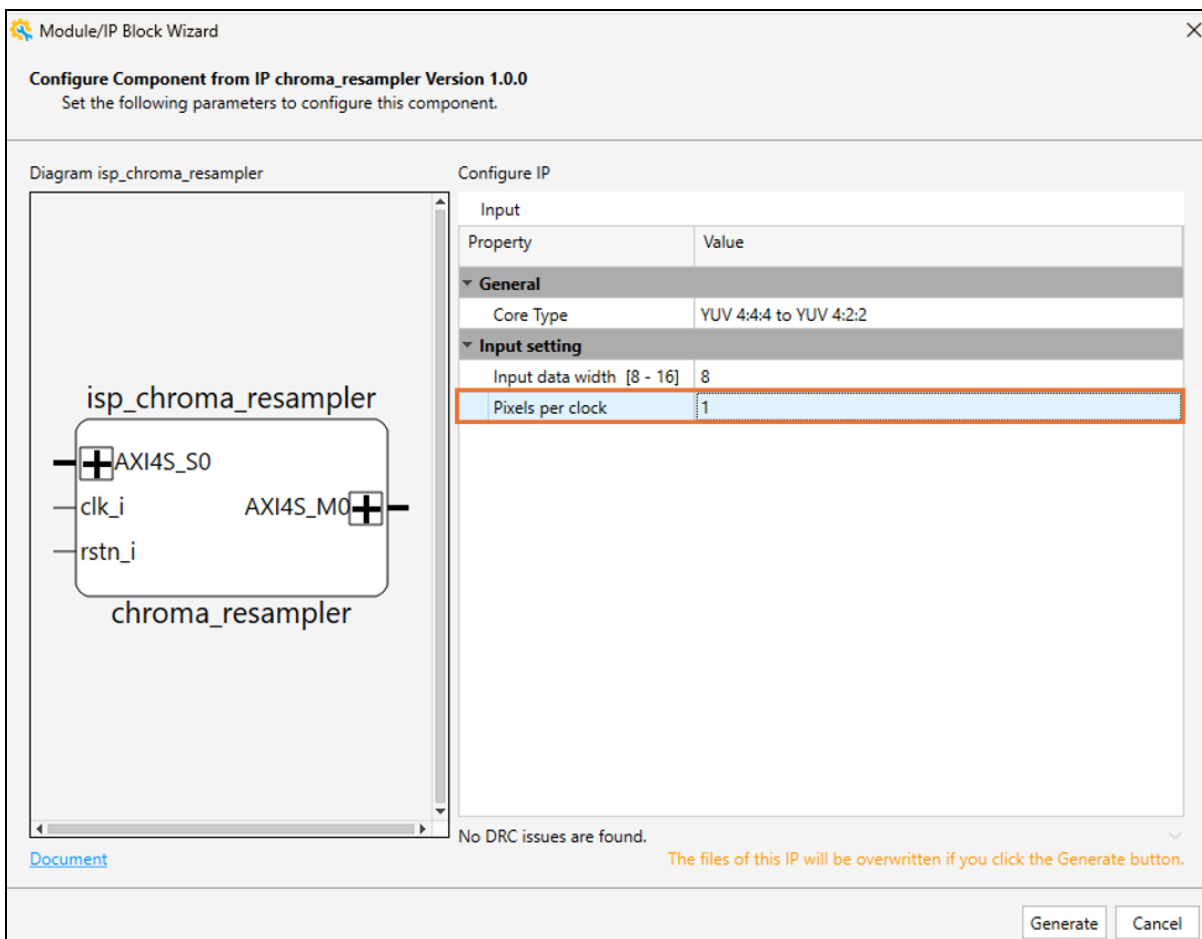


Figure 7.13. Chroma Resampler Configuration

3. Click **Generate**.
4. After the generation is completed, click **Finish** in the **Check Generated Result** window (Figure 7.14).

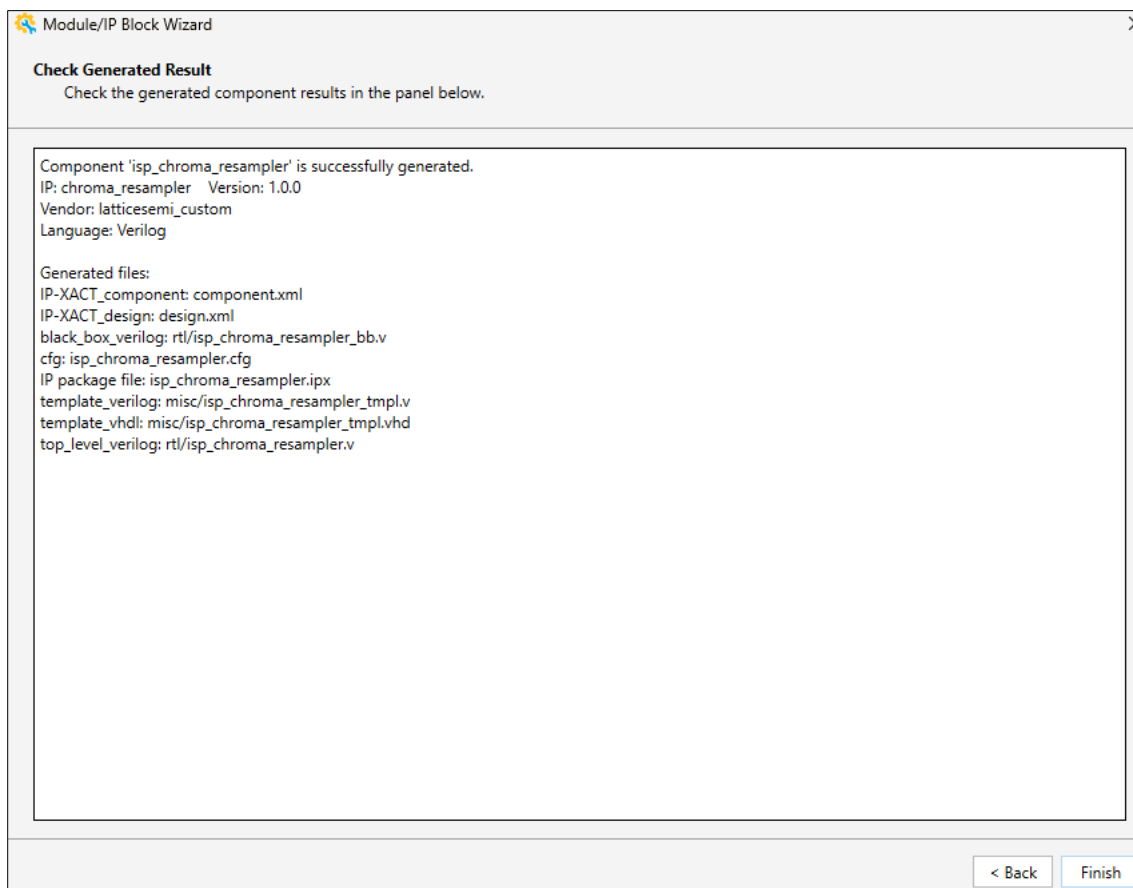


Figure 7.14. Chroma Resampler IP Check Generated Result Window

7.2.6. Configuring the Video to USB Bridge

To configure the Video to USB Bridge, follow these steps:

1. Double-click the unified_video_to_usb_inst block to open the Module/IP Block Wizard.
2. Under the **Input setting**, set the **Pixels per clock** to 1 (Figure 7.15).

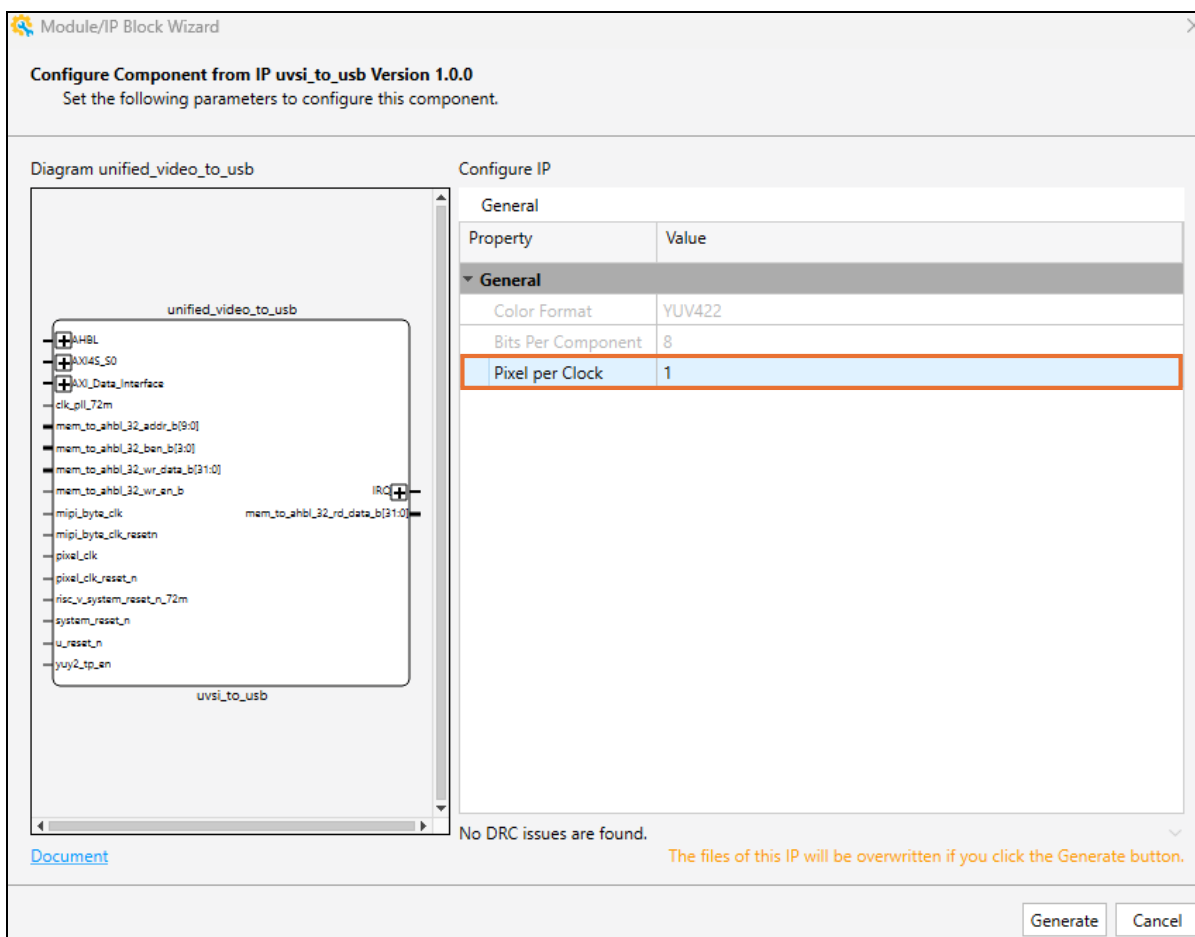


Figure 7.15. Video-to-USB Configuration

3. Click **Generate**.
4. After the generation is completed, click **Finish** in the **Check Generated Result** window (Figure 7.16).

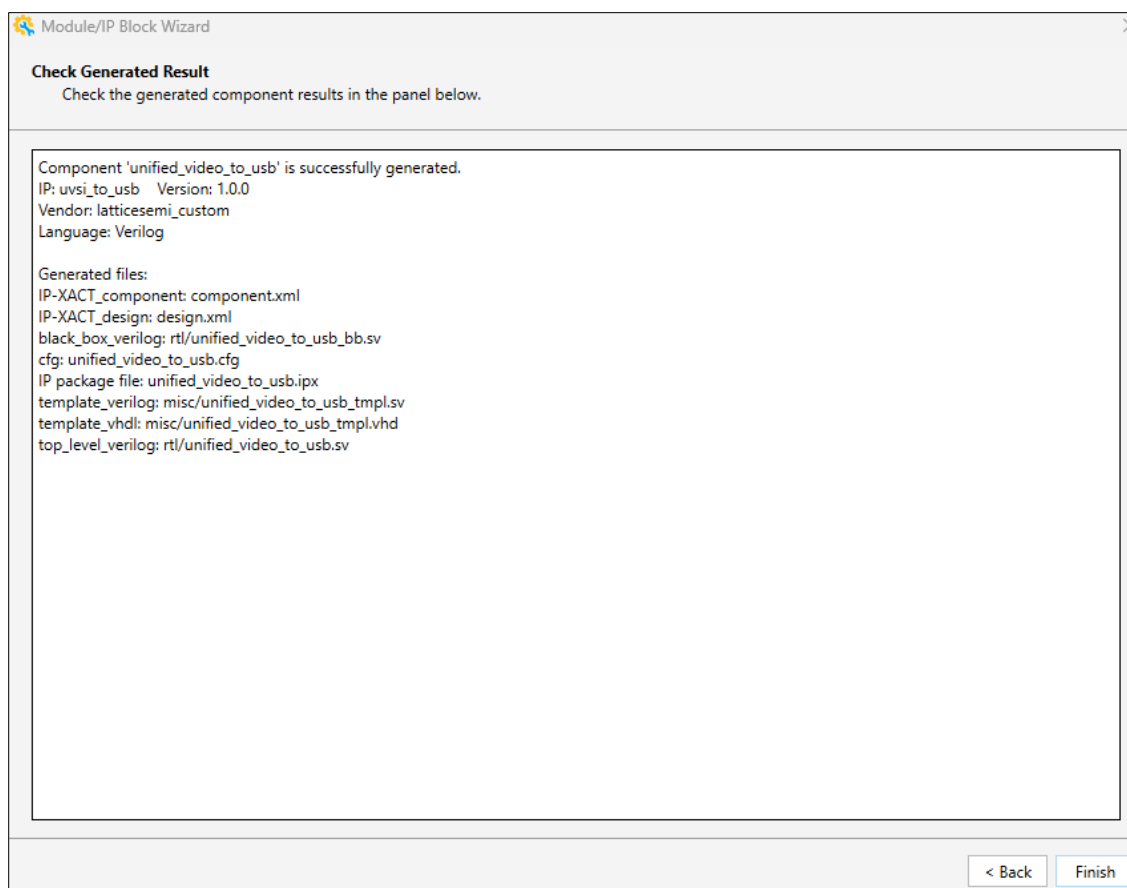


Figure 7.16. Video-to-USB Check Generated Result Window

After reconfiguration is completed, refer to the [Running the Reference Design](#) section to generate the memory image and bitstream. After the bitstream is generated, refer to the [Programming the FPGA](#) section to program the bitstream into the FPGA.

7.3. Steps to Enable YUV422 Test Pattern

1. Go to <Project Directory>/propelsdk/riscv_mc/src and open the main.c file.
2. Uncomment the following line, as shown in [Figure 7.17](#).

```
//Test pattern enable GPIO
// 0x1 - yuy2 tp en (YUV422 test pattern)
// lsc_8_write((TP_EN_GPIO_INST_BASE_ADDR + 0x4), 0x1);

enable_camera();
```

Figure 7.17. Firmware Code to Enable the YUV422 Test Pattern

3. Write the data value 0x1 to enable the YUV422 test pattern.
4. Rebuild the Propel SDK project.
5. After the .mem file is generated, refer to Step4 of the [Running Propel Builder and Radiant Project](#) section, to regenerate sysmem0_inst.

Note: YUV422 Test Pattern is only supported 1080p resolution.

8. Resource Utilization

Table 8.1 below shows the example configurations validated on the CrossLinkU-NX Evaluation board, including the resource utilization for the CrossLinkU-NX MIPI CSI-2 to USB UVC reference design. These configurations use the LIFCL-33U-9CTG104C device.

Table 8.1. Resource Utilization for CrosslinkU-NX MIPI CSI-2 to USB UVC Reference Design

MIPI CSI/DSI RX		ISP (Debayer, CCM, CSC, Chroma Resampler)		Resource Utilization			
Lane Count	Line Rate (Mbps)	BPC	PPC	LUT4	Registers	DSP	EBR
x2	592	8	2	19403	12974	36	22
x2	592	8	1	15628	10755	21	22

9. Debugging

9.1. Errors when Updating the SPI Flash

- If there is a verification error when programming the .bit file, try changing the TCK frequency by setting **TCK Divider Setting** to greater than three (3). Click **Program** to restart programming (Figure 9.1).

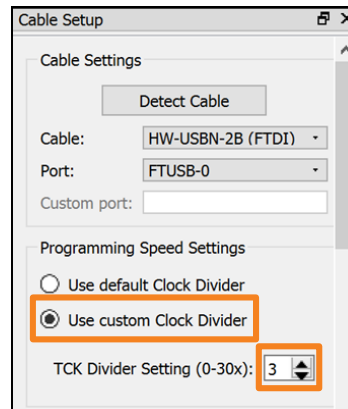


Figure 9.1. TCK Frequency Settings

- If the verification error issue still occurs, press and hold the **PROGRAMN** push button before clicking **Program**.
- If the device is not detected on Port: **FTUSB-0**, click **Detect Cable** and select Port: **FTUSB-1** (Figure 9.2).

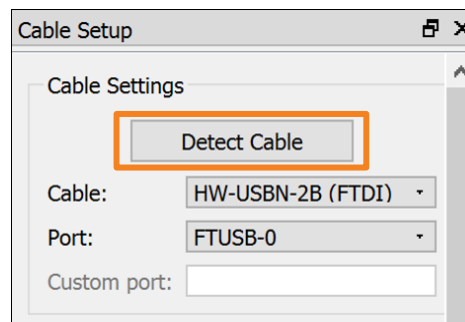


Figure 9.2. Port Selection

9.2. Timing Violations or Place and Route Failed

Timing violations may occur during compilation and may be caused by the Placement Iteration settings for Place and Route not being set optimally on the current machine. Besides, Place and Route may fail due to the signal being partially routed, as shown in Figure 9.3.

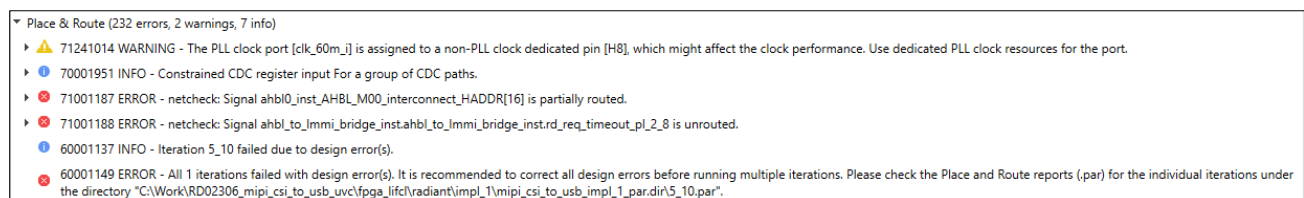


Figure 9.3. Place and Route Failed Message

To resolve these issues, follow these steps:

1. Go to **Project > Active Strategy > Place & Route Design Settings**.
2. In the settings window shown in Figure 9.4, change both **Placement Iteration** and **Placement Save Best Run** from 1 to 10. Select **Stop Once Timing is Met**.

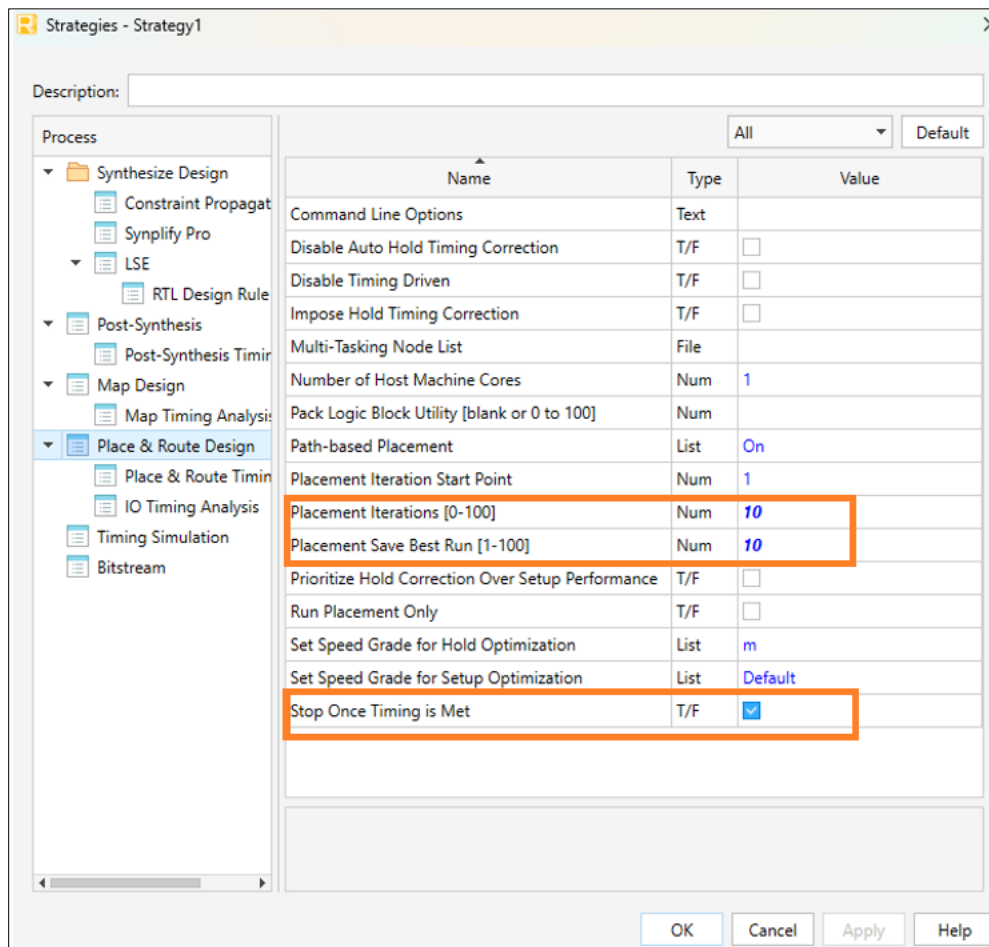


Figure 9.4. Place and Route Design Strategy Settings

3. Click **OK** to apply the changes.
4. Click **Export Files** to regenerate the bitstream.
5. If an iteration meets timing requirements, the Place and Route reports display the placement results.

References

- [CrossLinkU-NX Devices](#) web page
- [Lattice Radiant Timing Constraints Methodology \(FPGA-AN-02059\)](#)
- [CrossLinkU-NX](#) web page
- [USB 2.0/3.2 IP \(FPGA-IPUG-02237\)](#)
- [Lattice Propel Builder Software](#) web page
- [Lattice Radiant Software](#) web page
- [Lattice Insights](#) for Lattice Semiconductor training courses and learning plans

Technical Support Assistance

Submit a technical support case through www.latticesemi.com/techsupport.

For frequently asked questions, please refer to the Lattice Answer Database at www.latticesemi.com/Support/AnswerDatabase.

Revision History

Revision 1.2, March 2026

Section	Change Summary
All	Updated the reference design name from <i>CrossLinkU-NX USB Video Class</i> to <i>MIPI CSI-2 to USB UVC Reference Design</i> globally.
Introduction	- Updated Table 1.1. Summary of the Reference Design. - In the Features section, updated key features of the reference design to the current. - Updated the Naming Conventions section.
Directory Structure and Files	Updated Figure 2.1. Directory Structure .
Functional Description	- Updated Figure 3.1. Reference Design Block Diagram. - In the Design Components section: - removed the old PLL, MIPI Packet Decoder, Byte to Pixel, B2P to AXI Stream Conversion, AXI Stream to Parallel Conversion, YUV422 to UVC Bridge, IEBC in FIFO Interface, In Endpoint Buffer Manager, USB23 to AXI Bridge, RAW10 Test Pattern Generator, YUV422 Test Pattern In FIFO Bridge sections; - removed the old Figure 3.4. RISC-V MC IP Configuration and related description in the RISC-V Microcontroller section; - removed the old Figure 3.5. System Memory General Configuration, Figure 3.6. System Memory Port S0 Configuration, and Figure 3.7. System Memory Port S1 Configuration and related description in the System Memory section; - removed the old Figure 3.8. UART Configuration and related description in the UART section; - removed the old Figure 3.9. Hardware Version GPIO Configuration, Figure 3.10 Test Pattern Enable GPIO Configuration and related description in the GPIO section; - removed the old Figure 3.11 Lattice I2C Controller Configuration and updated the general description of the Lattice I2C Controller in the Lattice I2C Controller section; - removed the old Figure 3.16. APB Interconnect General Configuration and Figure 3.17. APB Interconnect Main Setting Configuration and related description in the APB Interconnect section; - removed the old Figure 3.18. APB-Lite to APB Converter Configuration and related description in the AHB-Lite to APB Converter section; - replaced the old RX DPHY section with the current MIPI CSI/DSI RX IP Core section; - replaced the old Debayer section with the current Debayer IP Core section; - removed the old Figure 3.25. Color Correction Matrix Configuration and updated the general description of the Color Correction Matrix IP Core in the Color Correction Matrix IP Core section; - removed the old Figure 3.26. Color Space Converter Input/Coefficient Configuration, Figure 3.27. Color Space Converter Output Configuration and updated general description of the Color Space Converter in the Color Space Converter IP Core section; - renamed the old AHB-Lite to Memory Bridge section to the current AHB-Lite to Native Memory Bridge section. - added Chroma Resampler and Video to USB Bridge sections. - In the Clocking Scheme section, removed the old Figure 3.29. Reference Design Clock Domain Block Diagram and added Figure 3.2. Clocking Scheme Overview. - In the Reset Scheme section, removed the old Figure 3.30 Reference Design Reset Scheme Diagram and added Figure 3.3. Reset Scheme Overview.
In Endpoint Buffer Manager Architecture	Removed this section.
USB23 AXI Manager to Memory Interface Bridge	Removed this section.
Signal Description	Renamed the section name from <i>UVC Reference Design Signal Description</i> to the current and updated Table 4.1. Primary I/O .
Building the Reference Design	Removed this section.

Section	Change Summary
Running the Reference Design	Added this section.
Running the Reference Design on Evaluation Board	Removed this section.
Implementing the Reference Design on Board	Added this section.
Customizing the Reference Design	Replaced the old Customizing the Reference Design section with detailed descriptions on the steps required to configure each IP and software for various demonstration setups.
Resource Utilization	Moved the resource utilization information to the current section and updated the data.
Debugging	Added this section.
References	Updated this section.

Revision 1.1, September 2025

Section	Change Summary
Functional Description	Updated Figure 3.1. Reference Design Block Diagram, Figure 3.10. Test Pattern Enable GPIO Configuration, Figure 3.29. Reference Design Clock Domain Block Diagram, and Figure 3.30. Reference Design Reset Scheme Diagram.
Building the Reference Design	- Updated Figure 7.12. Base Address Assignment. - In the Install IPs at Local from File section, added the following note: Note: for the latest version of the local IP files, contact Lattice Sales or Tech Support. - In the Opening Radiant Project section: - changed To create the FPGA bitstream file, open the Lattice Radiant software to To create the FPGA bitstream file, launch the Lattice Radiant software from Lattice Propel Builder; - changed The design should have no errors but there are 30 warnings due to some unused signals from some instance to The design should have no errors but there are 1074 warnings due to some unused signals from some instance; - updated Figure 7.15. Open the Radiant Design. - In the Generating Bitstream File section: - added If the Place & Route Design process encounters an out-of-memory error, reduce the number of host machine cores defined in Strategy1 > Place & Route Design from 7 to a smaller number; - added Figure 7.17. Place & Route Design Strategy.
LIFCL-33U Evaluation Board Programming	In the Radiant Programmer GUI Setup section, added <i>If the flash device is a GigaDevice flash, then the SPI Flash options are GD25LQ128E and 8-pin SOP. If the flash device is a Winbond flash, then the SPI Flash options are W25Q512JV and 8-pad WSON.</i>
Customizing the Reference Design	Added the Steps to Migrate from the FCCSP104 Package to the WLCSP84 Package section.

Revision 1.0, December 2024

Section	Change Summary
All	Production release.



www.latticesemi.com